

New Upper and Lower Bounds for Perfectly Secure MPC

Ivan Damgård¹, Shravani Patil², Arpita Patra², and Lawrence Roy¹

¹ Aarhus University, Denmark;

² IISc Bangalore, India;

Abstract. We consider perfectly secure MPC for n players and t malicious corruptions. We ask whether requiring only security with abort (rather than guaranteed output delivery, GOD) can help to achieve protocols with better resilience, communication complexity or round complexity. We show that for resilience and communication complexity, abort security does not help, one still needs $3t < n$ for a synchronous network and $4t < n$ in the asynchronous case. And, in both cases, a communication overhead of $O(n)$ bits per gate is necessary.

When $O(n)$ overhead is inevitable, one can explore if this overhead can be pushed to the preprocessing phase and the online phase can be achieved with $O(1)$ overhead. This result was recently achieved in the synchronous setting, in fact, with GOD guarantee. We show this same result in the asynchronous setting. This was previously open since the main standard approach to getting constant overhead in a synchronous on-line phase fails in the asynchronous setting. In particular, this shows that we do not need to settle for abort security to get an asynchronous perfectly secure protocol with overheads $O(n)$ and $O(1)$.

Lastly, in the synchronous setting, we show that perfect secure MPC with abort requires only 2 rounds, in contrast to protocols with GOD that require 4 rounds.

Table of Contents

1	Introduction	3
1.1	Our Contribution	3
1.2	Technical Overview	4
1.3	Network Model and Definitions	5
2	Lower Bounds	6
2.1	Some Intuition	6
2.2	Leaky Key Agreement Lowerbound	6
2.3	Leakage Auditor Model	9
2.4	Execution Transcripts	9
2.5	Communication Lower Bound, Synchronous Protocols	10
2.6	Communication Lower Bound, Asynchronous Protocols	12
3	Asynchronous Perfect MPC with Constant Online overhead	12
3.1	Preliminaries	13
3.2	The Outline	13
3.3	Online Phase Primitives	15
3.4	The Complete MPC Protocol	19
A	Proofs from the lower bounds section	22
B	VSS and MPC for Linear Functions when $n > 2t$	26
C	Asynchronous Perfect MPC with Constant Online Overhead	27
C.1	More Preliminaries	27
C.2	Preprocessing Phase	28
C.3	Online Phase	32
C.4	The Complete MPC Protocol	32
C.5	Proof of Theorem 3.11	33
D	2-Round Perfect MPC with Abort Security	37
D.1	2-Round Verifiable Secret Sharing	37
D.2	2-Round Multiplication	38
E	Proofs for 2-Round Perfectly-Secure Protocol with Abort Security	39
E.1	Proof of Theorem D.3	39
E.2	Proof of Theorem D.6	40

1 Introduction

In secure multiparty computation (MPC), a set of n mutually distrusting parties want to compute an agreed function on inputs held by the parties. The goal is to do this in such a way that the output is correct and is the only new information that is leaked; this must be true even if up to t of the parties are corrupted by an adversary.

In this paper we focus on the model where we want perfect information theoretic security, assuming secure point to point channels between each pair of parties. It is well known that in this model, perfect security is possible if and only if $t < n/3$ for a synchronous network, and $t < n/4$ for the asynchronous case. Moreover, a lower bound is known for the communication complexity [15], indicating that a communication overhead of $\Omega(n)$ bits per gate in the circuit is necessary when t is maximal³. Also, a matching upper bound is known [8,18,1]. Finally, regarding round complexity, it is known that 4 rounds are necessary and sufficient for perfectly secure MPC [6]. Based on all this, it may seem that the major questions in this model are answered.

However, a closer look reveals that this is not actually the case: the lower bounds on t are only known to hold for protocols with guaranteed output delivery (GOD), where all honest players always receive output. We could (and should) also consider the weaker property *security with abort*. In the weakest form of abort security, selective abort, the adversary gets the output and can decide whether each honest player gets output or will have to abort. In the following, selective abort is what we mean when talking about abort security. It is natural to ask whether requiring only abort security helps to achieve protocols with better properties, in terms of resilience, communication complexity or round complexity.

Surprisingly, to the best of our knowledge, these questions are open. Considering resilience first, one may note that the standard approach to prove $t < n/3$ is necessary for GOD is to invoke the well-known impossibility of broadcast when $t \geq n/3$. But this fails completely for abort security, as broadcast with abort is known to be possible for $t < n$ [17]. Furthermore, the known lower bound for communication complexity assumes GOD and a synchronous network, and no bounds are known for abort security and an asynchronous network. Finally, the known lower bound on round complexity also assumes GOD.

For completeness, we point out that the corresponding lower bound questions for statistical security are not very interesting, as it is easy to see that settling for abort security does not help in this setting. Namely, a non-trivial abort secure protocol must terminate normally and be secure if all players follow the protocol. So, a synchronous abort secure protocol implies a passively secure protocol for the same n and t , and it is well known that this requires $n > 2t$ even for passive corruption. For the asynchronous case, we note that if all players are honest, but we never hear from t of them, an abort secure protocol must terminate and be secure. But this scenario is indistinguishable from one where t players are passively corrupt and we never hear from t honest players, so again the protocol must terminate. This means we can obtain a (synchronous) secure protocol for $n - t$ players and t passive corruptions, which requires $n - t > 2t$, or $n > 3t$.

Finally, on the upper bound front, it is natural to ask if something can be achieved using preprocessing: even if we cannot go below $O(n)$ bits per gate starting from scratch, perhaps we can, assuming preprocessing? Here, the idea is that the protocol executes in two phases: a preprocessing phase where the inputs and function to compute are not known, but where some correlated randomness is produced, and then this randomness is “consumed” in an on-line phase where the function is computed. The hope is that this phase can be more efficient because the correlated randomness is available. This question on preprocessing was answered very recently for synchronous communication: a GOD protocol exists where the preprocessing has overhead $O(n)$ bits per gate, whereas the on-line phase has overhead $O(1)$ [20]. However, no similar solution is known for the asynchronous case, in particular it is not clear if we would need to settle for abort security to get such a result.

1.1 Our Contribution

In this paper, we settle all of the above questions. Our take-away message is in two parts: first, requiring only abort security does not help to achieve better resilience or communication complexity for general

³ To be precise, the bound says that there exist functions where perfect security requires communication of $\Omega(nS)$ bits where S is the input size. However, the functions in question have linear size circuits, so this translates to $\Omega(n)$ bits per gate for these circuits – but also for circuits in general, assuming that the protocol applies “the same” approach to any circuit, as is the case for all known general constructions.

MPC, but it does help significantly in terms of round complexity, and for resilience, if we settle for computing linear functions only. Second, in the off-line/on-line model, it is possible to get overheads $O(n)$ and $O(1)$ even in the asynchronous setting, and with GOD guarantee. In more detail:

- We show that $t < n/3$ is necessary for perfect abort security in general on a synchronous network, and $t < n/4$ is necessary in the asynchronous case.
- On the other hand, we show that Verifiable Secret Sharing (VSS) and secure computation of *linear* functions can be done with perfect abort security assuming only $t < n/2$.
- We show a communication lower bound indicating that an overhead of $\Omega(n)$ bits per gate is necessary to achieve perfect abort security when t is maximal, i.e., $n = 3t + 1$ in the synchronous case and $n = 4t + 1$ in the asynchronous case.
- We present a GOD protocol with preprocessing for an asynchronous network, where the preprocessing phase has communication overhead $O(n)$ bits per gate and the on-line phase has overhead $O(1)$ ⁴.
- We show that two rounds of communication are necessary and sufficient for perfect security with abort.

1.2 Technical Overview

Our lower bounds on resilience and communication share the same proof technique, which is new to the best of our knowledge. We start by assuming an n -party protocol π of the type we want to rule out, π is assumed to do a simple non-linear computation such as an OT from one party to another. We then devise a way to execute π by $n - 1$ parties (that are assumed to follow the protocol), where one party P from π is emulated in a special way by the other players: each time a party expects to get a message m from P , the party chooses a random bit string to play the role of m . This may result in π aborting, but the parties can discard such executions. Note that in case π does not abort, its input/output behaviour is correct, by perfect security against a corrupt P . Also, the probability that π terminates is non-zero. This is because there is a small but non-zero probability that the random messages correspond to P behaving honestly, and π must terminate without aborting if all players are honest. Finally, conditioned on the emulated P being honest, we can invoke perfect security against semi-honest corruption of one of the $n - 1$ remaining parties. Based on this, we argue that the $n - 1$ -party protocol accomplishes something that is not possible.

We show that VSS and linear MPC can be done with abort for honest majority by observing that the standard VSS based on bivariate polynomials becomes abort secure if one requires that no players complain about their shares; and if one requires in reconstruction that all players are on the same polynomial or else we abort. Then a linear function can be evaluated by VSS-ing the inputs and evaluating the function on the shares. Details can be found in appendix B.

We show that MPC with abort security can be done in two rounds by modifying an existing 2-round protocol with passive security. By adding some checks in the last round and aborting if they fail, we ensure that results are correct if we terminate. Details can be found in appendix D.

Finally, we consider our asynchronous protocol with $O(1)$ communication overhead per gate in the on-line phase. For this, we apply the well-known strategy of preparing in the preprocessing a number of packed Beaver triples, where a triple consists of 3 random secret-shared vectors $\mathbf{a}, \mathbf{b}, \mathbf{c}$ such that $\mathbf{a} * \mathbf{b} = \mathbf{c}$ where $*$ refers to coordinate-wise multiplication. Here, we are using packed secret-sharing, i.e., Shamir where the degree of the polynomial is increased to preserve privacy even though a vector rather than a single value is shared. However, shares are still single values. We also prepare a number of shared pairs of vectors of form $(\mathbf{a}, L(\mathbf{a}))$ where L is a linear function. This is useful in the on-line phase for rerouting values between layers of the circuit to compute. Based on existing techniques from [16,20,2], it is relatively straightforward to accomplish this with overhead $O(n)$ per preprocessed gate.

The main technical challenge occurs in the on-line phase. Given the preprocessing material, the communication in the on-line phase essentially reduces to opening a sequence of shared vectors. Doing this in the naive way where all parties send shares to all other parties introduces an overhead we cannot afford. Even though the use of packed secret sharing saves a factor of $\Theta(n)$, sending to all parties still gives a factor n overhead.

⁴ The result for the on-line phase assumes a certain SIMD-like structure of the circuit to compute. Using existing results, we can transform a circuit into an equivalent one with the right structure at the expense of a logarithmic overhead factor. See more details within.

The standard solution to this problem is to not send shares to all players, but only to a single player P_{king} who reconstructs locally and sends information back to all players. Intuitively this saves a factor $\Theta(n)$ and so can give us $O(1)$ overhead (here, we are ignoring for the moment how to verify what P_{king} does). A second problem is that packed secret sharing only allows coordinate-wise or SIMD-style multiplication and so, for general circuits, one needs to reorder values inside shared blocks. This (and other problems) can be solved if one can apply securely a linear function L to a shared block. Skipping many details, this can be done by having P_{king} apply L to the block she reconstructs.

These observations are the basis for prior work in the synchronous case [20]. Our goal is to get $O(1)$ overhead in the on-line phase for an asynchronous network. For this, several techniques from the synchronous protocol in [20] can be reused. A major challenge, however, is that the technique of using a single P_{king} for reconstruction does not work anymore. We may never hear from P_{king} , even if she is honest. Note that we cannot afford to wait, as we cannot distinguish between a single corrupt player and an honest player whose message is delayed. Assigning $t + 1$ or even all parties to all play the role of P_{king} works but reintroduces an overhead of $\Theta(n)$.

We propose instead the following solution: assume we can arrange it such that we can process $m = \Theta(n)$ shared blocks $(\mathbf{a}_1, \dots, \mathbf{a}_m)$ at the same time and that the same function L is to be applied to all of them. The idea is now to compute a Reed-Solomon style encoding $(\mathbf{b}_1, \dots, \mathbf{b}_n)$ of $(\mathbf{a}_1, \dots, \mathbf{a}_m)$, also in shared form. Since both the encoding and the sharing are linear, this only requires local computation. We then let each party play the role of P_{king} with respect to $\mathbf{b}_i$, i.e., players send shares of $\mathbf{b}_i$ to P_i who reconstructs, applies L , and sends back to the players shares of $L(\mathbf{b}_i)$ ⁵. When these shares are returned, players hold (in shared form) a Reed-Solomon encoding of the desired vector $(L(\mathbf{a}_1), \dots, L(\mathbf{a}_m))$, possibly with errors corresponding to the corrupt players. Since we can wait for $n - t$ players to return, we are guaranteed $3t + 1$ results with t errors which means decoding will work. Note that the vector to decode will be given in shared form, but it turns out that the players can apply decoding to the set of shares they hold. It will be the case that each such set of shares is a Reed Solomon codeword with at most t errors. This is because codewords are characterized by the property that a certain set of linear combinations of the entries (the syndrome) is 0. This property is preserved when applying a linear mapping to codewords, and this is what happens when all parties apply L to what they hold and then secret-share the result.

Now, since we are processing $\Theta(n)$ blocks in one go, this compensates for the cost paid by involving all n players so we still have $O(1)$ overhead. Finally, the decoding automatically corrects the errors introduced by corrupt players, so we need no special procedures for tracking down errors and eliminating players as required in [20].

To apply our solution, we need that the circuit is wide enough and has a sufficiently regular structure so that the same function L can be applied to all blocks coming out of a given layer of the circuit. The circuit may have this SIMD-style structure already, but it can also be ensured in general. Either because we evaluate the same circuit in parallel several times, or one can apply the technique from [13] to rewrite the circuit to an equivalent circuit that is a log factor larger, but has the required regular structure.

1.3 Network Model and Definitions

We consider synchronous and asynchronous network models, where the parties in $\mathcal{P} = \{P_1, \dots, P_n\}$ are connected via pairwise private and authenticated channels. Additionally, for some of our protocols we assume the availability of a broadcast channel, which allows a party to send an identical message to all the parties. The distrust among the parties is modeled as a centralized, computationally unbounded adversary that can corrupt up to t out of the n parties. When the network is synchronous, every message sent by any party is delivered within a fixed, known time bound Δ . Additionally, all the messages are delivered in the same order they are sent in. Whereas in the asynchronous network, messages are delivered with an arbitrary but finite delay. Moreover, the messages may be delivered in an arbitrary order. The only guarantee is that the messages get delivered eventually. This is modeled by a scheduler which decides on the sequence of message deliveries, and the scheduler itself is assumed to be controlled by the adversary.

⁵ The reconstruction will work, even in the case of packed secret sharing which uses degree larger than t implying that hearing from $n - t$ parties is not necessarily enough to decode to the right polynomial. Using so-called on-line decoding [10] one can detect if enough shares from honest parties are available and wait if not.

2 Lower Bounds

In this section, we show that perfect secure MPC with abort requires $t < n/3$ in the synchronous setting, and $t < n/4$ in the asynchronous setting.

We then prove a communication lower bound. It essentially says that realizing string oblivious transfer for strings of length ℓ with perfect abort security for $n = 3t + k$ parties and t corruptions requires that the total communication complexity is at least $\ell(n - 1)/k$, so when t is maximal, i.e., $k = 1$ we get a factor $\Omega(n)$ overhead. This is the same (type of) bound that was already known for perfectly secure protocols with GOD. Since string OT has a linear size (binary) circuit, we can also conclude that for this family of functions, the communication overhead to get perfect abort security must be a factor $\Omega(n)$ per multiplication gate. Since all known general MPC protocols do “the same” for every circuit, we expect the same overhead factor to be necessary in general.

For the asynchronous case, we get the same bound, but for $n = 4t + k$.

2.1 Some Intuition

We first give an intuitive explanation of the main ideas behind the proofs.

For the communication lower bound, it turns out to be enough to consider a 4 player protocol π for players A, B, C and D , and assume that π implements a string OT from A to B with perfect abort security and is secure against 1 corruption. So, A inputs two strings of length ℓ , and B has a selection bit that determines if she receives the first or the second string.

We then construct a 3-party protocol that is executed by A, B and C only where each time they expect a message from D , they will choose a uniformly random string and pretend this was what D sent. We assume all three follow the protocol. This “randomly emulated” D will most of the time not follow the protocol but because π is secure against a corrupt D , its input/output behaviour is correct every time it does not abort. Moreover there is a small but non-zero probability that the random messages correspond to D behaving honestly. In such a case, the protocol will not abort, and we can argue that the transcript of the communication between A and B contains no information on the string B receives because the protocol is secure against passive corruption of A or of B if D is honest.

We then repurpose the protocol to be a key agreement protocol between A and B with C acting as a helper, where the key is the string B receives in the OT. This is (with some non-zero probability) secure even if the adversary sees the communication between A and B . The intuition is now that this can only be the case if C communicates a lot, otherwise there is no way to get the entire string from A to B without the adversary learning anything.

This way we get a lower bound on the communication and this actually also implies the result on resilience: we argue we get a contradiction if C does not communicate (enough), but if we have only 3 players and one corruption this corresponds to C not communicating at all, which of course leads to a contradiction.

The result for the asynchronous case follows quite easily from the synchronous result by observing that if we have an additional player and 1 corruption, the last player could have its message delayed indefinitely, so if the protocol still works (as it should if it is asynchronously secure), we are back in the previous setting.

In the rest of this section we formalize this intuition. This is harder than it might seem at first, the main difficulty is that the 4-party protocol might terminate without aborting even if D was not honest, and in that case we have no privacy guarantees against A or B .

2.2 Leaky Key Agreement Lowerbound

We will reduce everything to a simple information theoretic lowerbound on key agreement extension, i.e., using a pre-existing key of length ℓ' to agree on a random key of length ℓ , where $\ell > \ell'$. This lowerbound will hold even in the presence of leakage and aborts, as long as it is possible for the protocol to complete without leakage. We slightly strengthen the notion of leakage here, by allowing the simulator to program what key is sampled in the case where it is leaked. This simplifies the argument a bit later on, by not requiring the key that is sampled to look random when it gets leaked.

Functionality 2.1: $\mathcal{F}_{AC}$ - Authenticated Channel Functionality

Whenever a message (m, P') is received from a party P :

1. Send (m, P, P') to $\mathcal{A}$.
2. Receive a bit a from $\mathcal{A}$.
3. If $a = 0$, send m to P' .

Functionality 2.2: $\mathcal{F}_{\text{PSK}}^{\ell'}$ - Pre-shared Key

The functionality is parameterized by a key length ℓ' .

1. Sample $k' \leftarrow \{0, 1\}^{\ell'}$.
2. Send k' to Alice.
3. Receive a bit a from $\mathcal{A}$.
4. If $a = 0$, send k' to Bob.

Functionality 2.3: $\mathcal{F}_{\text{LKA}}^{\ell}$ - Leaky Key Agreement

The functionality is parameterized by a message length ℓ .

1. Receive the leakage request $(l, L) \in \{0, 1\} \times \{0, 1\}^{\ell}$ from $\mathcal{A}$.
2. If $l = 0$ then sample $k \leftarrow \{0, 1\}^{\ell}$; else set $k = L$.
3. Send k to Alice.
4. Receive a bit a from $\mathcal{A}$.
5. If $a = 0$, send k to Bob.

Lemma 2.4. *Let π be a synchronous protocol that perfectly realizes $\mathcal{F}_{\text{LKA}}^{\ell}$ between honest parties Alice and Bob, using an authenticated insecure channel and ℓ' bits of correlated randomness. Then $\ell' \geq \ell$, or else it is impossible for SIM_{π} to send $l = a = 0$ to $\mathcal{F}_{\text{LKA}}^{\ell}$.*

In order to prove this, we need to define a suitable measure of correlation between the states of Alice and Bob. It must ignore the probabilities of leakage or abort. We define a notion that we call common max-information, which measures the max-entropy of everything known to them both.

Definition 2.5. *Let P be a probability distribution, and let $(X, Y) \in \mathcal{X} \times \mathcal{Y}$ be a pair of random variables defined over P . The common max-information $C_P(X; Y)$ of the random variables X and Y is defined as the maximum $\log_2(N)$ such that there exist functions $f: \mathcal{X} \rightarrow [N] \cup \{\perp\}$ and $g: \mathcal{Y} \rightarrow [N] \cup \{\perp\}$ where*

$$\Pr_{(X,Y) \leftarrow P} [f(X) = g(Y) \vee f(X) = \perp \vee g(Y) = \perp] = 1$$

and

$$\forall z \in [N]. \Pr_{(X,Y) \leftarrow P} [f(X) = z = g(Y)] > 0.$$

We will suppress the distribution P and write $C(X, Y)$ when P is unimportant or is clear from context. We will need the following basic properties to bound the common max-information of Alice's and Bob's states.

Lemma 2.6 (Symmetry). $C(X, Y) = C(Y, X)$

Lemma 2.7 (Upper bound by max-entropy).

$$C(X, Y) \leq |\log_2(\text{supp}(X))|$$

Proof. To satisfy the second condition, $f(X)$ must have support $[N]$ for $N = 2^{C(X,Y)}$. Since f is deterministic we have $|\text{supp}(f(X))| \leq |\text{supp}(X)|$. $\square$

Lemma 2.8 (Data-processing inequality). *For any randomized function $u: \mathcal{X} \rightarrow \mathcal{U}$,*

$$C(u(X), Y) \leq C(X, Y)$$

Proof. Let $f(U)$ and $g(Y)$ be some functions that satisfy Definition 2.5 for $U = u(X)$ and Y . For all $x \in \mathcal{X}$, let $f'(x)$ be most probable outcome of $f(u(x))$ other than $\perp$. If $f(u(x))$ is $\perp$ with probability 1, let $f'(x) = \perp$ instead. For any $(x, y) \in \text{supp}(X, Y)$, if there are two possible outcomes $u_1 \neq u_2$ for $u(x)$

then it must be that either $f(u_1) = g(y) = f(u_2)$, $f(u_1) = \perp$, $f(u_2) = \perp$, or $g(y) = \perp$. Otherwise, there would be positive probability of violating the first condition in Definition 2.5. Therefore,

$$\Pr_{(X,Y) \leftarrow P; U \leftarrow u(X)} [f'(X) = f(U) \vee f(U) = \perp \vee g(Y) = \perp] = 1. \quad (1)$$

This implies the first condition of Definition 2.5 on (f', g) : when $g(Y) \neq \perp$ then either $f'(X) = f(U)$, which implies the condition as it is satisfied by (f, g) , or $f(U) = \perp$ and so either $f'(X) = f(U')$ for some other U' (leading to the previous case) or $f'(X) = \perp$ (satisfying the condition directly).

Next, for any $z \in [N]$, there must be some (u_1, y) such that $f(u_1) = z = g(y)$ and there is positive probability that $u_1 = u(X) \wedge y = Y$. Since $z \neq \perp$, by (1) we have $f'(X) = z$ in this event. Therefore the second condition of Definition 2.5 is also satisfied. $\square$

Lemma 2.9 (One-sided conditioning). *For any event $E \subseteq \mathcal{X}$ that has positive probability,*

$$C_{P|E}(X, Y) \leq C_P(X, Y)$$

Proof. Given $f(X)$ and $g(Y)$ satisfying Definition 2.5 for $P | E$, define $f'(X)$ to be $f(X)$ if $X \in E$, and otherwise $f'(X) = \perp$. Then $f'(X)$ and $g(Y)$ satisfy Definition 2.5 for P . $\square$

We are now ready to prove our lowerbound on the correlated randomness needed to realize $\mathcal{F}_{\text{LKA}}^\ell$.

Proof of Lemma 2.4. Let P be the distribution of the computation trace and communication transcript of Alice and Bob running π , with access to pre-shared key $\mathcal{F}_{\text{PSK}}^{\ell'}$. Let the states of Alice and Bob in round i be S_A^i and S_B^i , respectively, and let r be the number of rounds in π . Without loss of generality, we can assume that $\mathcal{F}_{\text{PSK}}^{\ell'}$ gives Alice and Bob their pre-shared key $k' \in \{0, 1\}^{\ell'}$. When Alice and Bob get this pre-shared key, this is the randomness in their states, so we have $C_P(S_A^0, S_B^0) = \ell'$. Let T be the random variable describing Alice and Bob's communication transcript. Let T^* be in the support of T , and let T^i be the event that Alice and Bob's communication matches with T^* for rounds up to (and including) i . The event T^r is equivalent to $T = T^*$.

Next, we show by induction that for all rounds $i \leq r$, $C_P(S_A^i, S_B^i) \leq \ell'$. We already have the base case. For the induction case, we must prove that $C_{P|T^i}(S_A^i, S_B^i) \geq C_{P|T^{i+1}}(S_A^{i+1}, S_B^{i+1})$ for all $i \in [0, r]$. In each round, Alice and Bob can sample randomness and communicate. By Lemma 2.8, sampling local randomness cannot increase the common max-information. There is a unique message m_A^{i+1} that Alice sends to Bob for event T^{i+1} to occur, so Alice sending this message can be interpreted as first conditioning on this message, then modifying Bob's state to include m_A^i . Neither can increase the common max-information: the former by Lemma 2.9, and the latter by Lemma 2.8. A similar argument applies to Bob's messages to Alice.

Consider the keys (k_A, k_B) output by Alice and Bob. (Set a party's output key to $\perp$ if that party aborts.) These are a local function of the party's final states (S_A^r, S_B^r) , so by Lemma 2.8 we have $C_{P|T=T^*}(k_A, k_B) \leq \ell'$.

Next, we change to the ideal world to lower bound this common max-information. Let P' be the distribution given by running $\mathcal{F}_{\text{LKA}}^\ell$ and SIM_π , which contains random variables for Alice and Bob's communication transcript T (by SIM_π 's simulation of $\mathcal{F}_{\text{AC}}$), the key $k = k_A = k_B$ output by Alice and Bob (again, set a key to $\perp$ if the party aborts), and the bits l and a by which SIM_π requests leakage or abort from $\mathcal{F}_{\text{LKA}}^\ell$. Now consider the distribution $P' | T = T^*$, and let $f(k_A) = k_A$ and $g(k_B) = k_B$. These satisfy the first condition of Definition 2.5, as they must agree if no abort occurs. If it is possible that $T = T^*$, $l = 0$, and $a = 0$ all hold simultaneously, then they also satisfy the second condition (with $N = 2^\ell$), as when $l = 0$ the transcript T and abort bit a are both independent of k , so every (k, k) is in the support of the distribution for all $k \in \{0, 1\}^\ell$. Therefore, $C_{P'|T=T^*}(k_A, k_B) = \ell$.

Putting this together, if there is some transcript T^* that is possible for SIM_π to produce at the same time as it chooses $l = a = 0$, then $C_{P'|T=T^*}(k_A, k_B) = \ell$. However, we showed in the real world that $C_{P|T=T^*}(k_A, k_B) \leq \ell'$. Since T , k_A , and k_B are all given to the environment, $P | T = T^*$ and $P' | T = T^*$ must define identical distributions over k_A and k_B . Therefore,

$$\ell' \geq C_{P|T=T^*}(k_A, k_B) = C_{P'|T=T^*}(k_A, k_B) = \ell,$$

unless it is impossible for SIM_π to complete without leaking or aborting. $\square$

2.3 Leakage Auditor Model

We would like to analyze the OT protocol using an OT functionality that allows for leakage, similarly to $\mathcal{F}_{\text{LKA}}^\ell$. Unfortunately, for protocols where a party might be corrupt, modeling this kind of leakage is not so straightforward. Specifically, we must rule out protocols that just randomly choose a party, then just send all the inputs to this party and trust them to do the real computation. Such protocols have a positive probability of not leaking (i.e., of happening to pick an honest party). We rule out this kind of trivial protocol by requiring the leakage to be well-defined in the real world, independent of who is corrupted.

To model this, we add an extra party P_{audit} , who is always honest, but can only receive messages and cannot send any. Because P_{audit} cannot send messages, it cannot help the protocol succeed, so we do not count P_{audit} in the party count n or the threshold t , nor do we count the messages sent to P_{audit} as counting towards the communication complexity. P_{audit} 's only job is to output a bit $l \in \{0, 1\}$, indicating whether leakage occurred. This keeps the simulator honest in some sense, because it forces the choice of leakage made by the simulator to correspond to some real world event that is independent of at least semi-honest corruptions.

Here is our leaky OT functionality in this model.

Functionality 2.10: $\mathcal{F}_{\text{LOT}}^\ell$ - Leaky Oblivious Transfer Functionality

The functionality is parameterized by a message length ℓ .

1. Receive two strings $m_0, m_1 \in \{0, 1\}^\ell$ from P_1 .
 2. Receive the choice bit $b \in \{0, 1\}$ from P_2 .
 3. If P_2 is corrupt, send m_b to $\mathcal{A}$.
 4. Receive the leakage bit $l \in \{0, 1\}$ from $\mathcal{A}$.
 5. Send the leakage bit l to P_{audit} .
 6. Compute leakage $L = \begin{cases} (m_0, m_1, b) & \text{if } l = 1 \\ \perp & \text{if } l = 0 \end{cases}$.
 7. Send L to $\mathcal{A}$.
 8. Receive the abort bit a from $\mathcal{A}$.
 9. If $a = 0$ output m_b to P_2 . Otherwise, output “abort” to P_2 .
-

Note that any OT protocol without leakage can be converted into a protocol for $\mathcal{F}_{\text{LOT}}^\ell$, by adding a dummy P_{audit} who always outputs 0, and having the simulator always send $l = 0$ to $\mathcal{F}_{\text{LOT}}^\ell$.

2.4 Execution Transcripts

We use the transcript ts to refer to information that is visible to the environment⁶, and so must match between the real and ideal worlds. In addition to the views of the corrupt parties, ts contains at least the following information:

abort(ts): The abort bit a sent received by $\mathcal{F}_{\text{LOT}}^\ell$. A value of 1 indicates that the simulator requested an abort, while a value of 0 indicates that the protocol completed successfully.

leak(ts): The leakage bit l sent received by $\mathcal{F}_{\text{LOT}}^\ell$. The value 1 means that the simulator requested $\mathcal{F}_{\text{LOT}}^\ell$ to leak all inputs.

SC_{P,P'}(ts): Total size of all messages sent from P to P' . That is, this is the sum of n over all messages (n, P, P') sent by the simulator (pretending to be $\mathcal{F}_{\text{SC}}$) to $\mathcal{A}$.

TC_{P,P'}(ts): Total size of all messages sent from P to P' or from P' to P . Equals $\text{SC}_{P,P'}(\text{ts}) + \text{SC}_{P',P}(\text{ts})$.

SC_P(ts): Total sender communication of party P . Equals the sum of $\text{SC}_{P,P'}(\text{ts})$ over all P' (excluding P_{audit}).

TC_P(ts): Total communication of party P . Equals the sum of $\text{TC}_{P,P'}(\text{ts})$ over all P' (excluding P_{audit}).

TC(ts): Total communication of the entire protocol. Equals the sum of $\text{SC}_P(\text{ts})$ over all P , or equivalently half the sum of $\text{TC}_P(\text{ts})$ over all P (excluding P_{audit}).

⁶ Assuming a dummy adversary that passes on all information.

We call $\mathbf{ts}$ a “possible transcript” if there exist some inputs to the protocol π such that when all parties behave honestly it is possible for this $\mathbf{ts}$ to be produced. This is equivalent whether you take $\mathbf{ts}$ to be produced by in the real world or the ideal world, as perfect security implies that $\mathbf{ts}$ will have the same distribution for both.

2.5 Communication Lower Bound, Synchronous Protocols

In this section, we lower bound the communication of realizing the string OT functionality $\mathcal{F}_{\text{LOT}}^\ell$, in the selective abort setting, from secure channels $\mathcal{F}_{\text{SC}}$.⁷ In fact, we prove a statement that is stronger than required for our purpose — even if the protocol is allowed to sometimes leak to the adversary, whenever a leak does not occur the communication complexity is still lower bounded. This strengthening comes for free, as when we replace one of the parties with random behavior there will be a (very high) probability of leaking to the adversary.

Functionality 2.11: $\mathcal{F}_{\text{SC}}$ - Secure Channel Functionality

Whenever a message (m, P') is received from a party P :

1. Send $(|m|, P, P')$ to $\mathcal{A}$.
 2. Receive a bit a from $\mathcal{A}$.
 3. If $a = 0$, send m to P' .
-

Theorem 2.12. *Let π be a synchronous protocol for $n = 3t + k$ parties and t malicious corruptions that perfectly realizes $\mathcal{F}_{\text{LOT}}^\ell$ from $\mathcal{F}_{\text{SC}}$. Then for all possible transcripts $\mathbf{ts}$ of π , if $\text{abort}(\mathbf{ts}) = \text{leak}(\mathbf{ts}) = 0$ then every subset X of k parties in π that includes at most one of P_1 and P_2 must always have total communication $\sum_{P \in X} \text{TC}_P(\mathbf{ts}) \geq 2\ell$.⁸*

Averaging over all subsets X of k parties excluding P_2 , this gives a lower bound on communication of the entire protocol:

Corollary 2.13. *Let π be a synchronous protocol for $n = 3t + k$ parties and t malicious corruptions that perfectly realizes $\mathcal{F}_{\text{LOT}}^\ell$ from $\mathcal{F}_{\text{SC}}$. Then in every possible transcript $\mathbf{ts}$ of π such that $\text{abort}(\mathbf{ts}) = \text{leak}(\mathbf{ts}) = 0$, the total communication $\text{TC}(\mathbf{ts})$ is at least $\ell \cdot (n - 1)/k$.*

It also implies that string OT cannot be implemented with perfect abort security if $n \leq 3t$. Namely, if we had a protocol π doing this, say for $n = 3t$, we can construct a protocol π' with $3t + 1$ players by adding a dummy player to π that never communicates. π' clearly contradicts Theorem 2.12, so we have:

Corollary 2.14. *No protocol can perfectly realize $\mathcal{F}_{\text{LOT}}^\ell$ in the synchronous setting from secure channels with $n \leq 3t$ parties and t malicious corruptions while supporting a possible transcript $\mathbf{ts}$ such that $\text{abort}(\mathbf{ts}) = \text{leak}(\mathbf{ts}) = 0$.*

Our approach to proving Theorem 2.12 will be to reduce from simpler functionalities, eventually reaching one that we can lower bound information theoretically. First, we show that a lowerbound for the case of $n = 4$ and $t = 1$ implies the general case.

Lemma 2.15. *Let π be a synchronous protocol as in Theorem 2.12, and let X be a subset of k parties such that $\{P_1, P_2\} \not\subseteq X$. Then there exists a protocol π' for $n' = 4$ parties and $t' = 1$ malicious corruptions that also perfectly realizes $\mathcal{F}_{\text{LOT}}^\ell$. Additionally, there exists a party P'_i of π' such that, for each possible transcript $\mathbf{ts}$ of π , there exists a possible transcript $\mathbf{ts}'$ of π' such that $\text{abort}(\mathbf{ts}') = \text{abort}(\mathbf{ts})$, $\text{leak}(\mathbf{ts}') = \text{leak}(\mathbf{ts})$, and $\text{TC}_{P'_i}(\mathbf{ts}') \leq \sum_{P \in X} \text{TC}_P(\mathbf{ts})$.*

Proof. Define four the parties $P'_1, \dots, P'_4$ of π' , where each part P'_j will emulate a subset S_j of the n parties in π . Let the party corresponding to X be P'_i , where $i = 1$ if $X \in P_1$, $i = 2$ if $X \in P_2$, and $i = 3$ otherwise. Define $S_i = X$, and define the three other sets $\{S_j\}_{j \neq i}$ be an arbitrary partition of the remaining $n - k$ parties into three subsets of size t , such that $P_1 \in S_1$ and $P_2 \in S_2$. Then define π' to have each P'_j emulate the parties in S_j in the protocol π , with the inputs of P'_1 and P'_2 being provided

⁷ Note that there is only one party with output in $\mathcal{F}_{\text{LOT}}^\ell$, so selective and unanimous abort are equivalent.

⁸ We defer the proof of this theorem to the end of this subsection.

to the emulated P_1 and P_2 , respectively, and the output of the emulated P_2 being output by P'_2 . P'_{audit} in π' will behave the same as P_{audit} in π , with all messages sent by some $P_j \in S_{j'}$ of π to P_{audit} being sent by $P'_{j'}$ to P'_{audit} in π' .

Every S_j satisfies $|S_j| \leq t$, so a single corruption in π' can be simulated by corrupting up to t parties in π . Therefore, we can simulate π' by running the simulator for π , corrupting S_j if P'_j is corrupted, forgetting all communication between two parties in the same subset, and otherwise passing all messages through. Since all communication between P'_i and P'_j in π' appears as communication between parties in S_i and S_j in π , we have $\text{TC}_{P'_i}(\text{ts}') \leq \sum_{P \in X} \text{TC}_P(\text{ts})$.⁹ $\square$

Next, we show that an $n = 4$ protocol with malicious security implies an $n = 3$ protocol with semi-honest security. Since a simulator for malicious security can lie about the inputs of corrupted parties, malicious security does not always imply semi-honest security. We first show that for $\mathcal{F}_{\text{LOT}}^\ell$ malicious security always implies semi-honest security, so we can always assume that the $n = 4$ protocol has semi-honest security when reducing to the $n = 3$ case.

Lemma 2.16. *Let π be a synchronous protocol that perfectly realizes $\mathcal{F}_{\text{LOT}}^\ell$ from secure channels for $n = 4$ and $t = 1$ malicious corruptions. Then π also perfectly realizes $\mathcal{F}_{\text{LOT}}^\ell$ for $t = 1$ semi-honest corruptions.*

Proof. The only way in which semi-honest corruptions can make simulation harder is that they do not allow the party's inputs to be controlled by the simulator. To simulate semi-honest corruptions, we ask SIM_π to simulate the case where a party P_s is maliciously corrupted, but make P_s behave identically to an honest P_s . We will show that either leakage occurs, making semi-honest simulation trivial, or the input SIM_π extracts for P_s must be identical to P_s 's real input.

This is trivial for $s = 3$, as P_3 has no input. For $s = 1$, if SIM_π extracts some $(m'_0, m'_1) \neq (m_0, m_1)$, then there will be some b such that $m'_b \neq m_b$. If P_2 inputs b (and SIM_π does not get to see whether it did) then the environment can distinguish, as P_2 will output m'_b in the ideal world and m_b in the real world. Therefore, SIM_π will always extract $(m'_0, m'_1) = (m_0, m_1)$. Similarly, if $s = 2$ and SIM_π extracts some $b' \neq b$, then if P_1 inputs uniformly random (m_0, m_1) then the simulator's view will only contain $m_{b'}$, and be independent of m_b . Therefore, the simulator has low probability of simulating a protocol execution that causes P_2 (who is behaving honestly, even though it was maliciously corrupted) to output m_b . This would allow the environment to distinguish, because in the real world P_2 would output m_b . $\square$

Lemma 2.17. *Let π be a synchronous protocol that perfectly realizes $\mathcal{F}_{\text{LOT}}^\ell$ from secure channels for $n = 4$ and $t = 1$ corruptions (supporting both malicious and semi-honest corruptions), let P_i be a party (for some $i \in \{1, \dots, 4\}$), and let ts be a possible transcript of π where $\text{abort}(\text{ts}) = \text{leak}(\text{ts}) = 0$. Then there exists a protocol π' for $n' = 3$ parties and $t' = 1$ semi-honest corruptions that also perfectly realizes $\mathcal{F}_{\text{LOT}}^\ell$, such that there exists a party $P'_{i'}$ of π' (for some $i' \in \{1, 2, 3\}$) and a possible transcript ts' of π' , such that $\text{abort}(\text{ts}') = \text{leak}(\text{ts}') = 0$ and $\text{TC}_{P'_{i'}}(\text{ts}') \leq \text{TC}_{P_i}(\text{ts})$.*

The proof of this lemma is found in Appendix A. It is based on the technique mentioned in the introduction, where the three parties in π' run π by emulating directly three parties in π while replacing messages from the fourth party by uniformly random messages. Since there is a small but non-zero probability that this corresponds to the fourth party behaving honestly, we get an implementation of $\mathcal{F}_{\text{LOT}}^\ell$, albeit with a small (but non-zero) probability of not leaking.

Next, we reduce to a two party problem: key agreement extension between Alice and Bob (both honest) in the presence of an eavesdropper. It is well known that two-party OT implies key agreement, but this argument relies on having a single communication channel the eavesdropper can see. Since we instead have a three-party OT, with communication channels between each pair of parties, we have to do something different. We assume that Alice and Bob have a pre-shared key proportional to the communication complexity along one of the three channels, so that this channel can be masked by a one-time pad. Another channel will be handled by merging two parties, so that communication between these two parties is local to either Alice's or Bob's mind. This leaves just a single channel to eavesdrop on. From here we can use a variation of the usual proof that OT implies key agreement. Later, we will use Lemma 2.4 to show that the pre-shared key must be big enough to one-time pad Alice's entire message, so this in turn lower bounds the communication of along each channel in the OT protocol. The proof of the following lemma is found in Appendix A.

⁹ Equality fails because communication between two parties in S_i will appear in π , but not in π' .

Lemma 2.18. *Let π be a synchronous protocol that perfectly realizes $\mathcal{F}_{\text{LOT}}^\ell$ from secure channels for $n = 3$ and $t = 1$ semi-honest corruptions, and let $i \neq i' \in \{1, \dots, 3\}$ and $\ell' \geq 0$. Then there exists a protocol π' that perfectly realizes $\mathcal{F}_{\text{LKA}}^\ell$ between parties Alice and Bob, using an authenticated insecure channel $\mathcal{F}_{\text{AC}}$ and ℓ' bits of correlated randomness $\mathcal{F}_{\text{PSK}}^{\ell'}$. If it is possible that π neither leaks nor aborts while the channel between P_i and $P_{i'}$ has total communication at most ℓ' , then it is also possible that π' neither leaks nor aborts. Additionally, if there is a possible transcript ts of π where $\text{TC}_{P, P'}(\text{ts}) \leq \ell'$ and $\text{abort}(\text{ts}) = \text{leak}(\text{ts}) = 0$, there exists a possible transcript ts' of π' such that $\text{abort}(\text{ts}') = \text{leak}(\text{ts}') = 0$.*

Finally, we put all the above lemmas together into a complete proof.

Proof of Theorem 2.12. From the precondition, let π be a synchronous protocol for $n = 3t + k$ parties and t malicious corruptions, let ts be a possible transcript of π such that $\text{abort}(\text{ts}) = \text{leak}(\text{ts}) = 0$, and let $X \not\subseteq \{P_1, P_2\}$ be an arbitrary subset of k parties whose communication we wish to lowerbound. Assume for contradiction that $\sum_{P \in X} \text{TC}_P(\text{ts}) < 2\ell$. Use Lemma 2.15 to get a four-party malicious protocol π' for $\mathcal{F}_{\text{LOT}}^\ell$ where some party P'_i has total communication at most that of X . That is, there is a possible transcript ts' of π' such that $\text{abort}(\text{ts}') = \text{leak}(\text{ts}') = 0$, and $\text{TC}_{P'_i}(\text{ts}') \leq \sum_{P \in X} \text{TC}_P(\text{ts}) < 2\ell$. Then Lemma 2.17 turns π' into a three-party semi-honest protocol π'' for $\mathcal{F}_{\text{LOT}}^\ell$, in which some party P'_j matches the total communication of P'_i . That is, there exists there exists a party P''_j and a possible transcript ts'' of π'' , such that $\text{abort}(\text{ts}'') = \text{leak}(\text{ts}'') = 0$ and $\text{TC}_{P''_j}(\text{ts}'') \leq \text{TC}_{P'_i}(\text{ts}') < 2\ell$. For each $k \in \{1, 2, 3\} \setminus \{j\}$, Lemma 2.18 turns π'' into a two-party protocol π''' for $\mathcal{F}_{\text{LKA}}^\ell$ that uses an authenticated channel and ℓ' bits of correlated randomness, where $\ell' = \text{TC}_{P''_j, P''_k}(\text{ts}'')$. Lemma 2.4 then shows that $\ell' \geq \ell$. Finally, there are two possible k for each j , so

$$2\ell > \text{TC}_{P''_j}(\text{ts}'') = \sum_{k \neq j} \text{TC}_{P''_j, P''_k}(\text{ts}'') \geq \sum_{k \neq j} \ell = 2\ell. \quad \square$$

2.6 Communication Lower Bound, Asynchronous Protocols

The standard argument of delaying t parties arbitrarily shows a corresponding lower-bound in the asynchronous setting.

Theorem 2.19. *Let π be an asynchronous protocol for $n = 4t + k$ parties and t malicious corruptions that perfectly realizes $\mathcal{F}_{\text{LOT}}^\ell$ from secure channels. If it is possible for π to complete with total communication ℓ' while not leaking or aborting, and while some subset of t parties disjoint from $\{P_1, P_2\}$ is adversarially delayed, then $\ell' \geq \ell \cdot (n - t - 1)/k \geq \ell \cdot \frac{3}{4}(n - 1)/k$.*

Proof. Let D be the set of adversarially delayed parties from the theorem's assumptions. Define a protocol π' that runs π over a synchronous network on $n' = 3t + k$ parties, excluding all parties from D as being adversarially delayed. Then π' perfectly realizes $\mathcal{F}_{\text{LOT}}^\ell$ from secure channels, and it is possible for π' to complete with total communication ℓ' without aborting or leaking. By Corollary 2.13, we then have that $\ell' \geq \ell \cdot (n' - 1)/k = \ell \cdot (n - t - 1)/k$. $\square$

Corollary 2.20. *No protocol can perfectly realize $\mathcal{F}_{\text{LOT}}^\ell$ in the asynchronous setting from secure channels with $n \leq 4t$ parties and t malicious corruptions, unless π always leaks or aborts when t parties are adversarially delayed.*

3 Asynchronous Perfect MPC with Constant Online overhead

We design a perfect MPC with malicious security for the optimal resilience of $t < n/4$ in an asynchronous network. The protocol has a communication complexity of $\mathcal{O}(n)$ per multiplication gate in the preprocessing phase while it offers $\mathcal{O}(1)$ online cost per gate¹⁰. We achieve the aforementioned communication complexity using the techniques of packed secret sharing, which allows us to evaluate a bunch of $\mathcal{O}(n)$ gates of the same kind simultaneously. Below we begin with a brief preliminary, followed by an outline, the online-phase primitives and the MPC protocol. The details of preprocessing phase appear in Appendix C.

¹⁰ The result for the online phase is considering SIMD style of circuit evaluation. Alternatively, using [13], we can transform the circuit into the required structure at the expense of logarithmic (in the circuit size) overhead.

3.1 Preliminaries

Our protocols are defined over a field $\mathbb{F}$ such that $|\mathbb{F}| > 3n/2$ and we denote its elements by $\{-n/2, \dots, -1, 0, 1, \dots, n\}$. We use k to denote the number of gates that are “packed” together, or equivalently, evaluated simultaneously. Further, we use degree- d packed Shamir sharing, where $d = t + k - 1$ to ensure privacy of k packed secrets against an adversary that can corrupt up to t parties. In our protocols, the degree of sharing after computation can go up to $d + k - 1 = t + 2k - 2$. To allow honest parties to reconstruct, we have to ensure $n - t > t + 2k - 2$. Hence, when $t < n/4$, we can set $k \leq t/2 + 1$. Unless stated otherwise, we use $k = t/2$ in our protocols. Throughout, we represent by $[v|j]_d$, a degree- d Shamir sharing of a value v via a degree- d polynomial $f(x)$ where $f(-j) = v$ for some $-j \in \mathbb{F}$. Further, we use $[\mathbf{v}]_d$ to denote the packed Shamir sharing of a vector $(v_1, \dots, v_k)$ via a degree- d polynomial $f(x)$ where $f(-j) = v_j$ for $j \in \{1, \dots, k\}$. The term $\mathbf{e}_i$ denotes the unit vector $(0, \dots, 0, 1, 0, \dots, 0)$ which is a k -length vector with 1 in the i th position and 0 in all the remaining positions.

3.2 The Outline

The Preprocessing Phase: In this phase, we generate the input-independent data such as Beaver triples, which is then utilised to evaluate the circuit on the actual inputs of parties in the subsequent online phase. This is particularly useful since we can afford a cost of $\mathcal{O}(n)$ communication per gate in the preprocessing phase to generate correlated data that ensures an efficient online phase. We first abstract out the requirements of the preprocessing phase for a simpler exposition and elaborate on the precise requirements subsequently. Our protocol primarily requires the following types of preprocessing correlations to be generated:

1. **Packed Beaver triples:** We require shared triples of the form $([\mathbf{a}]_d, [\mathbf{b}]_d, [\mathbf{c}]_d)$ where $c_i = a_i b_i$ for all $i \in \{1, \dots, k\}$. For this, we follow the approach of [16, 20] outlined below:
 - (a) First, parties generate standard degree- t shared Beaver triples $([a_i|i]_t, [b_i|i]_t, [c_i|i]_t)$ for each $i \in \{1, \dots, k\}$ using an existing protocol such as [2, Theorem 6.4 with Section 6.1].
 - (b) Following this, parties convert a group of k such triples $\{(a_i, b_i, c_i)\}_{i \in \{1, \dots, k\}}$ into a single packed triple $([\mathbf{a}]_d, [\mathbf{b}]_d, [\mathbf{c}]_d)$ locally using techniques from [16]. Specifically, parties can compute $[\mathbf{a}]_d = \sum_{i=1}^k [\mathbf{e}_i]_{k-1} * [a_i|i]_t$, where $[\mathbf{e}_i]_{k-1}$ is the default degree- $(k-1)$ Shamir sharing of the unit vector $\mathbf{e}_i$.
2. **Packed linear transformation pair:** We generate pairs of the form $([\mathbf{r}]_d, [L(\mathbf{r})]_d)$, where L is a linear transformation to be applied on the vector $\mathbf{r}$. At a high level, this will be used to perform a linear transformation of “packed” output wires of every layer in the circuit to match the required sequence for the “packed” input wires of the subsequent layer. We adapt the protocol from [20] in the synchronous network model to suit our setting. Omitting most of the details, the primary challenge we encounter in migrating their protocol to the asynchronous setting is in reconstructing shared vector of k values or alternatively, establishing a default degree- $(k-1)$ sharing of these values without cost inflation. For instance, to reduce $[\mathbf{v}]_{d+k-1}$ to $[\mathbf{v}]_{k-1}$, their protocol simply reconstructs $\mathbf{v}$ to a P_{king} , who further shares $[\mathbf{v}]_{k-1}$ with a total cost of $\mathcal{O}(n)$. It is crucial to note that this degree reduction is only used for values that can be revealed publicly without hampering privacy. However, a naive public reconstruction is avoided to prevent $\Omega(n^2)$ communication cost, which is inevitable. However, the same technique cannot translate to the asynchronous network setting due to the following reason. It is impossible to distinguish between a corrupt P_{king} which does not distribute $[\mathbf{v}]_{k-1}$, and an honest P_{king} which is slow. This may result in an endless wait. On the other hand, using (at least) $t + 1$ parties as P_{king} will ensure liveness; however, this will result in cost inflation by a factor of n which we cannot afford. Hence, we require a new mechanism for performing such a degree reduction of values that can be revealed publicly, which comprises our primary contribution in this protocol. The rest of the protocol follows from [20], and we discuss it in the relevant sections.
3. **Packed degree reduction pair:** We require pairs of the form $([\mathbf{r}]_d, [\mathbf{r}]_{d+k-1})$. During the online phase, $\mathbf{r}$ acts as a random pad for a packed Shamir shared vector of k values, say $\mathbf{x}$ to facilitate reducing the degree of the sharing polynomial. In particular, given the above pair and $[\mathbf{x}]_{d+k-1}$, parties can compute $[\mathbf{x} + \mathbf{r}]_{d+k-1}$ locally. Given that the elements of $\mathbf{r}$ now mask, or pad the elements of $\mathbf{x}$, revealing $\mathbf{x} + \mathbf{r}$ does not raise any privacy concerns. Thus, it can be publicly reconstructed, or even shared with a threshold lower than t . For instance, if we can ensure $[\mathbf{x} + \mathbf{r}]_{k-1}$ via some

procedure such as public reconstruction, then parties can locally compute $[\mathbf{x}]_d = [\mathbf{x} + \mathbf{r}]_{k-1} - [\mathbf{r}]_d$. Thus, the task of reducing the degree of a shared vector $\mathbf{x}$ from $(d + k - 1)$ to d while ensuring its privacy now reduces to a simpler task of reducing the degree of the masked vector $\mathbf{x} + \mathbf{r}$, which can be revealed publicly, from $(d + k - 1)$ to $(k - 1)$. This degree reduction required in the online phase reduces to the same as that required in the earlier case of generating linear transformation pairs.

Given that the operation to switch from $\{[x_i|i]_t\}_{i \in \{1, \dots, k\}}$ to $[\mathbf{x}]_d$ is local, the protocol in [20] maintains the invariant of degree- t sharing for each wire value instead of the packed sharing described above. The packed sharing of the required values is then computed locally on demand. In our protocol as well, we follow the same approach. For instance, for the packed linear transformation pair, instead of computing $([\mathbf{r}]_d, [L(\mathbf{r})]_d)$, we maintain $\{([r_i|i]_t, [L_i(\mathbf{r})|i]_t)\}_{i \in \{1, \dots, k\}}$ where $L_i(\mathbf{r})$ is the i th element of vector $L(\mathbf{r})$. Similarly, for the packed degree reduction pair, we maintain $\{[r_i|i]_t\}_{i \in \{1, \dots, k\}}$ which in turn allows us to compute $[\mathbf{r}]_d$ as described earlier. We can also compute $[\mathbf{r}]_{d+k-1} = \sum_{i=1}^{2k-1} [e_i]_{2k-2} * [r_i|i]_t$ using some additional dummy values $\{[r_i|i]_t\}_{i \in \{k+1, \dots, 2k-1\}}$.

The Online Phase: The online phase utilises the preprocessed data to evaluate the circuit on the actual inputs of parties as follows.

1. **Input:** Here, each party holding inputs distributes a degree- t Shamir sharing of each input. For this, we use the VSS protocol from [12] with a $(t + k, t)$ -degree bivariate polynomial to share k inputs simultaneously. If the same input is provided to multiple input wires of the circuit, then we require to perform a fan-out operation to copy the same value as many times as required. As observed in prior works such as [16, 20], fan-out can be viewed as a linear transformation operation on wire values. Suppose w.l.o.g. that a party has k inputs $\mathbf{x} = (x_1, \dots, x_k)$ such that it has shared $[x_i]_t$ for all $i \in \{1, \dots, k\}$. As described earlier, parties can locally obtain $[\mathbf{x}]_d$. Now let us say that x_1 is used as an input for 4 input wires, then as described in [16, 20], we effectively require to create some $\mathbf{y} = (x_1, x_1, x_1, x_1, \cdot, \dots, \cdot)$. It was observed that $\mathbf{y}$ can thus be viewed as some linear transformation L applied on the vector $\mathbf{x}$, that is, $\mathbf{y} = L(\mathbf{x})$. We discuss how linear transformation is achieved subsequently.
2. **Linear Transformation:** In addition to fan-out, linear transformations are also required for applying permutation on ‘packed’ wire values. This is because the values to be packed in a single sharing for evaluation of a layer of gates may not be in the appropriate position in the sharing. For instance, suppose a ‘packed’ gate in the subsequent layer requires the input $\mathbf{x} = (x_1, x_2, \cdot, \dots, \cdot)$, where x_1 and x_2 are inputs provided by P_1 and P_2 respectively. The procedure for packing k degree- t shared values locally into a degree- $(t + k - 1)$ allows for packing when we have $[x_1|1]_t$ and $[x_2|2]_t$ shared by P_1 and P_2 respectively. That is, when all the values to be packed together are shared at different positions from $\{-1, \dots, -k\}$ in their respective sharing. However, it is possible that P_1 and P_2 both shared their inputs at the same position in their respective sharing polynomial. Say $f(x)$ and $g(x)$ are the degree- t polynomials shared by P_1 and P_2 respectively, then we may have $f(-1) = x_1$ and $g(-1) = x_2$. That is, P_1 shared $[x_1|1]_t$, whereas P_2 shared its input as $[x_2|1]_t$, preventing the local packing operation. This issue can be resolved if k inputs shared by P_2 are permuted in order to obtain $[x_2|2]_t$ by replacing the -1^{th} position with some other input. Another caveat in the subsequent layers is that the output of a single gate may be used as an input to multiple gates in the subsequent layer. This requires a fan-out operation to be performed at the output of each layer in the circuit to ensure enough copies of a particular wire value. As observed in the prior works [16, 20], this reduces to a linear transformation on the packed wire values. Given that fan-out and permutation are specific types of a linear transformation operations, we discuss the more generic operation of transformation in the subsequent discussion. Again, similar to the earlier phase, we discuss the high level ideas first for a better exposition.

Thus, our goal is as follows: Given packed sharing $[\mathbf{x}]_d$ of some $\mathbf{x} = (x_1, \dots, x_k)$, we wish to obtain $[L(\mathbf{x})]_d$ where $L(\mathbf{x})$ is some linear transformation on the vector $\mathbf{x}$. For this, we make use of the linear transformation pair $([\mathbf{r}]_d, [L(\mathbf{r})]_d)$ generated during the preprocessing. Specifically, parties can locally compute $[\mathbf{x} + \mathbf{r}]_d$. Note that this value can be publicly revealed since the input $\mathbf{x}$ is now masked with the random vector $\mathbf{r}$ which is not known to any party, and further it can be permuted to obtain $L(\mathbf{x} + \mathbf{r})$. Equivalently, parties can also compute $[L(\mathbf{x} + \mathbf{r})]_{k-1}$ instead of reconstructing. Given this, parties can compute $[L(\mathbf{x})]_d = [L(\mathbf{x} + \mathbf{r})]_{k-1} - [L(\mathbf{r})]_d$. Our task is now reduced to

computing $[L(\mathbf{x} + \mathbf{r})]_{k-1}$ from $[\mathbf{x} + \mathbf{r}]_d$ of the vector $\mathbf{x} + \mathbf{r}$. That is, we again reduce our task to degree reduction of shared values.

Similar to the preprocessing phase, although we have described the ideas with degree- d shared values, the invariant we maintain is degree- t shares of every value. We obtain degree- d packed sharing by packing values on demand via local operations as described earlier.

3. **Computation:** This phase in turn consists of shared evaluation of two types of gates: addition and multiplication.
 - (a) Addition gates: These gates can be computed locally as long as the values to be added are shared in the correct position in packed Shamir sharing. The prior step of permutation ensures the required ordering of the values which are inputs to these gates. Hence, for a packed gate with k inputs x_i, y_i and output z_i , parties will hold $[x_i]_t$ and $[y_i]_t$ for all $i \in \{1, \dots, k\}$. They can locally compute $[z_i]_t = [x_i]_t + [y_i]_t$. Note that the same gate output z_i may be used as an input to many gates in the subsequent layers, and moreover, z_i may be required to be shared in some other position such as $[z_i]_t$ where $i \neq j$. To handle this fan-out and permutation requirement, parties execute the linear transformation protocol as described above after each layer of gate evaluation.
 - (b) Multiplication gates: Unlike the addition gates, these gates require parties to communicate with each other. Similar to the case of addition, for a packed gate with inputs x_i, y_i and output z_i , parties will hold $[x_i]_t$ and $[y_i]_t$ for all $i \in \{1, \dots, k\}$. Thus, parties can compute $[\mathbf{x}]_d$ and $[\mathbf{y}]_d$ locally and further, utilise one packed Beaver triple from the preprocessing phase to evaluate the gate. The evaluation itself follows Beaver's multiplication protocol, but for packed sharing, as in the prior works [16,20]. Thus given a triple $([\mathbf{a}]_d, [\mathbf{b}]_d, [\mathbf{c}]_d)$, parties first compute $[\mathbf{x} + \mathbf{a}]_d$ and $[\mathbf{y} + \mathbf{b}]_d$ which is required to be reconstructed. However, a public reconstruction would result in a communication of $\Omega(n^2)$ which we cannot afford. The work of [20] circumvents this problem by reconstructing to a designated P_{king} as described earlier. To prevent endless wait due to asynchrony in our case while ensuring the desired communication cost, we again follow our approach of degree reduction at this stage to obtain $[\mathbf{x} + \mathbf{a}]_{k-1}$ and $[\mathbf{y} + \mathbf{b}]_{k-1}$. While we follow the Beaver's multiplication protocol as in [20] for the rest of the computation, we encounter the same task of public reconstruction again for which we use another instance of our degree reduction protocol. The details are provided in Protocol 3.6. Similar to the case of addition, to handle the fan-out and permutation requirement for the input wires of the next layer gates, parties execute the linear transformation protocol.
4. **Output:** Finally, the output layer ensures that the output of the circuit is reconstructed to the designated party. For instance, suppose a party P_i has to receive the output corresponding to a wire v that is shared as per $[v]_t$ among parties. Parties send their respective share of $[v]_t$ towards P_i , who can robustly reconstruct the sharing polynomial using online error correction (OEC) [11] and recover v by evaluating the polynomial at $-i^{\text{th}}$ position. In fact, OEC can be used to reconstruct polynomials of higher degrees such as $d + k - 1$ used in our work and we discuss the protocol for reconstruction in the subsequent section.

3.3 Online Phase Primitives

Input Sharing In the online phase, parties begin by sharing their inputs to the circuit using a degree- t Shamir sharing. For this, parties use the VSS protocol from [12]. The functionality for VSS appears below, followed by our input sharing protocol. Protocol 3.2 has a communication complexity of $\mathcal{O}(n^4)$ elements over point-to-point channels to share $k = \mathcal{O}(n)$ secrets via degree- t Shamir sharing. However, the broadcast cost incurred in their protocol can be batched over many instances for better amortized complexity as mentioned in the below lemma.

Functionality 3.1: $\mathcal{F}_{\text{VSS}}$ - VSS Functionality

The functionality is parameterized by the set of corrupted parties $\mathcal{C} \subseteq [n]$.

1. The dealer sends a bivariate polynomial $S(x, y)$ to the functionality.
2. If $S(x, y)$ is of degree at most $2t$ in x and at most t in y , the functionality sends to the ideal adversary the shares $S(x, i), S(i, y)$ for each $i \in \mathcal{C}$ and send to each party P_j the polynomials $S(x, j), S(j, y)$. Otherwise the functionality does not terminate.

Protocol 3.2: Input Sharing Protocol

- **Input:** A party P_s holding k inputs to the circuit $\mathbf{x} = (x_1, x_2, \dots, x_k)$.
 - **The protocol:**
 1. P_s invokes the packed VSS of [12], $\mathcal{F}_{\text{VSS}}$, with a degree $(t + k, t)$ -degree bivariate polynomial to share $[x_j|j]_t$ for each $j \in \{1, \dots, k\}$.
-

Lemma 3.3. *Protocol 3.2 when run in the batching mode has a communication complexity of $\mathcal{O}(np + n^4)$ elements over point-to-point channels to share p secrets via degree- t Shamir sharing.*

It is possible that the same input is used in many different wires of the circuit. Thus, we require a fan-out operation after the layer of input gates. Since fan-out is a special case of linear transformation, we discuss it in the subsequent section.

Degree Reduction The degree reduction protocol allows parties holding a packed sharing of k secrets on a degree- $(d + \ell)$ polynomial, to obtain a degree- $(k - 1)$ sharing of the same secrets. In our work, we require degree reduction for two specific cases, when $\ell = 0$ and $\ell = k - 1$. We thus give a protocol for the latter case to reduce the degree from $d + k - 1$ to $k - 1$, and the one for the former case follows naturally by substituting $k = 0$. As already discussed, degree reduction is required in several protocols such as multiplication and linear transformation for obtaining the default degree- $(k - 1)$ sharing of masked vectors of k values. Designing a degree reduction protocol while ensuring $\mathcal{O}(1)$ communication per secret poses significant challenges in the asynchronous network where we cannot rely on a designated P_{king} as in the previous work [20].

At a very high level, our approach for mitigating this issue is to move away from reconstructing towards a single P_{king} , and instead, rely on all the parties to perform the degree reduction in a distributed manner. However, performing a reconstruction towards all the parties would immediately result in an increase in the communication cost. To handle this, instead of performing the degree reduction for a single vector of k values, we batch it over a set of k such vectors. This high level idea omits a lot of technicalities involved in our approach which we elaborate on below while describing the protocol.

At a conceptual level, one can think of our protocol to be an extension of the batched reconstruction from [14] over vectors of values. In our protocol, parties perform the degree reduction of k vectors say $\mathbf{u}^1, \dots, \mathbf{u}^k$, each having k components. At the onset of the protocol, parties hold degree- $(d + k - 1)$ sharing of each of these vectors and the goal is to obtain the default degree- $(k - 1)$ sharing of each. For this, parties expand these k sharings of degree- d each to n sharings, each of degree- d using linear error correcting codes that tolerate up to t errors. Further, parties open one sharing per party, which recovers k secrets from it and further distributes a degree- $(k - 1)$ sharing of these secrets. Finally, using the shares obtained from each party, parties correct t errors to obtain the shares corresponding to degree- $(k - 1)$ sharing of each $\mathbf{u}^1, \dots, \mathbf{u}^k$. We now describe why such an error correction is possible. Observe that k vectors $\mathbf{u}^1, \dots, \mathbf{u}^k$, each of length k , are encoded to form k codewords of length- n each. In matrix form, this corresponds to mapping a $(k \times k)$ matrix, whose i th column is $\mathbf{u}^i$, to a $(k \times n)$ matrix, say $\mathbf{P}$, where $\mathbf{P} = [\mathbf{P}(1), \dots, \mathbf{P}(n)]$ where $\mathbf{P}(i)$ is the i th column and where every row is a codeword.

In the protocol, the i th column vector $\mathbf{P}(i)$ is reconstructed towards party P_i by sending shares to P_i . Following this, P_i sends to every party P_j a linear combination of the k entries in its column $\mathbf{P}(i)$. This is implemented by Step 4 in Protocol 3.4, where it is described as P_i sharing $\mathbf{P}(i)$ among the parties with degree $k - 1$. Since we are secret-sharing without privacy here, this is equivalent to sending fixed linear combinations. Crucially, for each party P_j , this linear combination is the same across all the senders P_i . This way, P_j receives a fixed linear combination of the k rows of the matrix $\mathbf{P}$. This is a codeword of length- n , with up to t incorrect values obtained from corrupt parties. This is within the code's decoding capability (since the distance of the code is $n - k$ which is at least $3t + t/2$ since $k \leq t/2 + 1$). Hence, every party gets correct results and we do not rely on decoding being linear.

In case parties wish to obtain the degree reduced form of vectors with some linear transformation L applied on it (as is required in Protocol 3.8), each party locally applies L on the k secrets that it recovers

before computing degree- $(k - 1)$ sharing. The detailed protocol appears below without including the details for linear transformation since it follows naturally. It has a communication complexity of $\mathcal{O}(n^5)$ elements over point-to-point channels to reduce the degree of sharing of k vectors each with k values (total of $\mathcal{O}(n^2)$ values) from $d + k - 1$ to $k - 1$.

Protocol 3.4: Degree Reduction Protocol

- **Input:** Parties hold $\{[u^i]_{d+k-1}\}_{i=1,\dots,k}$ and the goal is to obtain $\{[u^i]_{k-1}\}_{i=1,\dots,k}$.
 - **The protocol:**
 1. Parties define a vector of k polynomials $P(x) = u^1 + u^2 \cdot x + \dots + u^k \cdot x^{k-1}$.
 2. Parties compute $[P(x)]_{d+k-1} = [u^1]_{d+k-1} + [u^2]_{d+k-1} \cdot x + \dots + [u^k]_{d+k-1} \cdot x^{k-1}$.
 3. Parties compute $[P(i)]_{d+k-1}$ and reconstruct the vector of k values, $P(i)$ to P_i for each $i \in \{1, \dots, n\}$.
 4. Each P_i shares $[P(i)]_{k-1}$ among all the parties using Protocol C.14.
 5. Parties run an instance of the ACS protocol to agree on a common set of $n - t$ parties which completed their sharing in the prior step. Let this set be denoted by **Com**.
 6. Parties apply the RS decoding procedure for recovering $[P(x)]_{k-1}$ on $[P(i)]_{k-1}$ for each $P_i \in \text{Com}$ instead of applying it directly on $P(i)$ values.
 7. Each P_i thus obtains $[P(x)]_{k-1} = [u^1]_{k-1} + [u^2]_{k-1} \cdot x + \dots + [u^k]_{k-1} \cdot x^{k-1}$ and computes its respective shares $\{[u^i]_{k-1}\}_{i=1,\dots,k}$.
-

Lemma 3.5. *Protocol 3.4 when run in the batching mode (with $\Omega(n^3)$ instances in parallel) has a communication complexity of $\mathcal{O}(p + n^5)$ elements over point-to-point channels to reduce the degree of sharing polynomials consisting of p values.*

We note that for the above idea to work, we want a party to verifiably generate degree- $(k - 1)$ sharing of k non-private values with constant overhead per value. In the following, we give a protocol for this purpose.

VSS without Privacy For degree reduction, we require parties to distribute a default degree- $(k - 1)$ sharing of a vector of k non-private values. We observe that the existing VSS protocol of [12] can be tweaked for this task. Specifically, [12] gives a VSS protocol considering a bivariate polynomial with degree $(2t, t)$ that allows parties to create $t + 1$ Shamir sharings simultaneously, each with a degree- t . Given that in the standard setting privacy has to be maintained against an adversary that can corrupt up to t parties, we can share 1 secret via each of these $t + 1$ sharings. Thus, we can share $t + 1 = \mathcal{O}(n)$ secrets while ensuring privacy. In contrast, for our case we require to share masked values for which privacy is not a concern, but ensuring the correctness of the degree is. By modifying the VSS of [12] to operate with degree- $(k - 1, k - 1)$ bivariate polynomial, we can precisely achieve this. That is, we can now create k Shamir sharings, each with degree- $(k - 1)$. Each of these k sharings of degree $k - 1$ can be used to share k secrets, thus allowing a total of $k^2 = \mathcal{O}(n^2)$ secrets to be shared simultaneously without privacy. Crucially, the communication cost of VSS in [12] is $\mathcal{O}(n^4)$ field elements for sharing $\mathcal{O}(n)$ secrets simultaneously, while at the same cost we can share $\mathcal{O}(n^2)$ values without privacy. More generally their protocol incurs a cost of $\mathcal{O}(np + n^4)$ for sharing p secrets with privacy whereas a cost of $\mathcal{O}(p + n^4)$ for sharing p secrets without privacy. In Appendix C, we describe the protocol from [12] with degree- $(k - 1, k - 1)$ bivariate polynomial for completeness.

Multiplication We follow the outline of the multiplication protocol from [20] for evaluating multiplication gates. Due to the invariant maintained in the protocol, parties hold degree- t Shamir sharing of each input wire value x_i, y_i for all $i \in \{1, \dots, k\}$ from which they can locally obtain $[x]_d, [y]_d$. Additionally, from the preprocessing phase, they also have such sharing of Beaver triples of the form $\{(a_i, b_i, c_i)\}_{i \in \{1, \dots, k\}}$ using which they can locally compute a degree- d sharing of a packed Beaver triple of the form (a, b, c) . Following this, as discussed earlier, evaluating a packed multiplication gate essentially reduces to performing degree reduction on shared values, either from degree d or $d + k - 1$, to degree $k - 1$. To achieve this, while ensuring a cost of $\mathcal{O}(1)$ in our network setting, we replace the P_{king} based approach

for degree reduction with our new protocol from the earlier section. The rest of the protocol remains the same and it appears below. It has a communication complexity of $\mathcal{O}(n^5)$ elements over point-to-point channels to evaluate $k^2 = \mathcal{O}(n^2)$ multiplication gates simultaneously.

Protocol 3.6: Multiplication Protocol

- **Input:** For k groups of k -packed multiplication gates in the same layer, for all $i \in \{1, \dots, k\}$ parties hold $\{[x_j^i|j]_t, [y_j^i|j]_t\}_{j=1}^k$. Parties also hold $\{([a_j^i|j]_t, [b_j^i|j]_t, [c_j^i|j]_t)\}_{j=1}^k$ and $\{[r_j^i|j]_t\}_{j=1}^{2k-1}$ from the preprocessing phase for each $i \in \{1, \dots, k\}$.
 - **The protocol:**
 1. Parties locally compute for each $i \in \{1, \dots, k\}$:
 - $[a^i]_d = \sum_{j=1}^k [e_j]_{k-1} * [a_j^i|j]_t$
 - $[b^i]_d = \sum_{j=1}^k [e_j]_{k-1} * [b_j^i|j]_t$
 - $[c^i]_d = \sum_{j=1}^k [e_j]_{k-1} * [c_j^i|j]_t$
 - $[x^i + a^i]_d = \sum_{j=1}^k [e_j]_{k-1} * [x_j^i|j]_t + [a^i]_d$
 - $[y^i + b^i]_d = \sum_{j=1}^k [e_j]_{k-1} * [y_j^i|j]_t + [b^i]_d$
 2. Invoke the degree reduction protocol (Protocol 3.4 for a batch of k multiplication gates in the same level simultaneously to obtain $[x^i + a^i]_{k-1}, [y^i + b^i]_{k-1}$ for all $i \in \{1, \dots, k\}$.
 3. Parties locally compute
 - $[z^i]_{d+k-1} = [x^i + a^i]_{k-1} * [y^i + b^i]_{k-1} - [x^i + a^i]_{k-1} * [b^i]_d - [y^i + b^i]_{k-1} * [a^i]_d + [c^i]_d$
 - $[r^i]_{d+k-1} = \sum_{j=1}^{2k-1} [e_j]_{2k-2} * [r_j^i|j]_t$
 - $[z^i + r^i]_{d+k-1} = [z^i]_{d+k-1} + [r^i]_{d+k-1}$
 4. Invoke the degree reduction protocol (Protocol 3.4 for a batch of k multiplication gates in the same level simultaneously to obtain $[z^i]_{k-1}$ for all $i \in \{1, \dots, k\}$.
 5. Parties compute $[z_j^i|j]_t = [z^i + r^i]_{k-1} - [r_j^i|j]_t$ for all $i \in \{1, \dots, k\}$.
-

Lemma 3.7. *Protocol 3.6 when run in the batching mode (with $\Omega(n^3)$ instances in parallel) has a communication complexity of $\mathcal{O}(p + n^5)$ elements over point-to-point channels to evaluate p multiplication gates simultaneously.*

Linear Transformation As discussed, evaluating a circuit using packed secret sharing involves computing linear transformations on k packed wire values that are shared among parties. Specifically, the linear transformation protocol allows parties to compute degree- d shares of $L(\mathbf{x})$, given the shares of $\mathbf{x}$ using some data from the preprocessing stage. We follow the approach of [20], where parties generate the pair $(\mathbf{r}, L(\mathbf{r}))$ during the preprocessing phase¹¹. As noted, the linear transformation operation involves reducing the degree of sharing of a vector of non-private values $\mathbf{x} + \mathbf{r}$ with the linear transformation L applied it. Specifically, it requires reducing the degree of $\mathbf{x} + \mathbf{r}$ from d to obtain a degree- $(k-1)$ sharing of $L(\mathbf{x} + \mathbf{r})$.

Once again, we rely on our degree reduction protocol, although with a caveat. Note that, in the case of linear transformations, we don't just reduce the degree of the vector $\mathbf{x} + \mathbf{r}$, but we also apply a linear transformation on it before reducing the degree. Going back to our degree reduction protocol, it works with a batch of k vectors simultaneously to ensure the desired communication cost. Given that we require to apply the linear transformation on values within each of these vectors during degree reduction, we need all the vectors for which the degree is being reduced to have the same linear transformation being applied. However, this may not be the case in a generic circuit. To circumvent this issue, we can rely on the approach of [13] wherein the circuit C to be computed is first compiled into a new circuit C' such that there is a set of $\log n$ linear transformations which get reused throughout the circuit. This allows us to batch the degree reduction protocol for the same linear transformation with the guarantee that at most $\log n$ different instances of such degree reduction may incur a higher cost in case sufficient values cannot be batched together. Another approach to tackle this is to simply execute our protocol for SIMD

¹¹ As in [20], instead of holding degree- d sharing of all the vectors $\mathbf{x}, \mathbf{r}, L(\mathbf{r})$, parties hold degree- t sharing of each of its components which is then locally converted to a degree- d sharing.

circuits, wherein instead of one execution, $m = \mathcal{O}(n^3)$ number of instances of the circuit are executed simultaneously. Here, the batching of degree reduction will happen for the same gates across m circuits. We describe our protocols assuming the latter, and a similar extension can be done for the former case as well. The protocol for linear transformation appears below assuming that the degree reduction occurs on a batch of vectors simultaneously. Specifically, the protocol has a communication complexity of $\mathcal{O}(n^5)$ elements over point-to-point channels to apply the same linear transformation L on k vectors each with k values (total of $\mathcal{O}(n^2)$ values).

Protocol 3.8: Linear Transformation Protocol

- **Input:** Parties hold $\{([r_j|j]_t, [L_j(\mathbf{r})|j]_t)\}_{j=1,\dots,k}$ from the preprocessing phase where $\mathbf{r} = (r_1, \dots, r_k)$, and $[x_j|j]_t$. The goal is to obtain $[L_j(\mathbf{x})|j]_t$ for all $j \in \{1, \dots, k\}$ where $\mathbf{x} = (x_1, \dots, x_k)$ and $L_j(\mathbf{x})$ is the j th component of $L(\mathbf{x})$.
 - **The protocol:**
 1. Parties compute $[\mathbf{x}]_d = \sum_{j=1}^k [e_j]_{k-1} * [x_j|j]_t$ and $[\mathbf{r}]_d = \sum_{j=1}^k [e_j]_{k-1} * [r_j|j]_t$ locally.
 2. Parties compute $[\mathbf{x} + \mathbf{r}]_d = [\mathbf{x}]_d + [\mathbf{r}]_d$ locally.
 3. Invoke the degree reduction protocol (Protocol 3.4) for a batch of k linear transformation gates simultaneously to obtain $[L(\mathbf{x} + \mathbf{r})]_{k-1}$.
 4. Parties compute $[L_j(\mathbf{x})|j]_t = [L(\mathbf{x} + \mathbf{r})]_{k-1} - [L_j(\mathbf{r})|j]_t$ locally.
-

Lemma 3.9. *Protocol 3.8 when run in the batching mode (with $\Omega(n^3)$ instances in parallel) has a communication complexity of $\mathcal{O}(p + n^5)$ elements over point-to-point channels to apply the same linear transformation L on vectors consisting of p values.*

3.4 The Complete MPC Protocol

In this section, we first provide the functionality for MPC, followed by the description of the complete MPC protocol as a composition of all the building blocks defined so far.

Functionality 3.10: AMPC – $\mathcal{F}_{\text{MPC}}$

The functionality is parameterized by a set of corrupted parties $\mathcal{C} \subseteq [n]$. Initialize the sets $S, H, \mathcal{C}' = \emptyset$. Initialize $x_i = 0$ for every $i \in \mathcal{C}$.

Input: Each P_i holds input $x_i \in \mathbb{F} \cup \{\perp\}$.

Common Input: An n -party function $f(x_1, \dots, x_n)$.

1. Upon receiving (Input, j, x_j) from an honest party P_j , if $j \notin H$ then add j to S .
 2. Receive from the adversary, the sets $H \subset [n] \setminus \mathcal{C}$ and $\mathcal{C}' \subseteq \mathcal{C}$ such that $|H| \leq |\mathcal{C}'| \leq t$. Also receive a set of inputs $\{(\text{Input}, i, x_i)\}_{i \in \mathcal{C}'}$.
 3. If $|S| < n - t$, then for each P_j with $j \in H$, the functionality sets $x_j = 0$ and updates $S = S \setminus H$.
 4. If $|S \cup \mathcal{C}'| \geq n - t$, then compute $(y_1, \dots, y_n) = f(x_1, \dots, x_n)$ and send y_i to P_i for every $i \in [n]$ and terminate.
-

We divide the MPC protocol in three phases, namely, the circuit transformation, preprocessing and online phase as described below.

(Circuit Transformation Phase)

- Parties first apply the deterministic circuit transformation algorithm from [19] to the circuit C with depth D to obtain a circuit C' which provides packed secret sharing friendliness¹². The resultant circuit C' is such that $|C'| = \mathcal{O}(|C| + k \cdot (n + D))$ and has the following properties:
 - The number of input gates and the number of output gates belonging to each party are a multiple of k each. Moreover, the number of addition gates and multiplication gates in every intermediate layer are also a multiple of k .

¹² Alternatively, the circuit could also be transformed using the algorithm from [13] leading to a circuit C' such that $|C'| = \mathcal{O}(|C| \log |C| + nD^2 \log^3 |C|)$ and $D' = \mathcal{O}(D \log^2 |C|)$

- The number of output wires of the input layer as well as every intermediate layer is a multiple of k . Similarly, the number of input wires of every intermediate layer and the output layer is also a multiple of k . Moreover, every output wire is used exactly once as the input wire in the next layer.
- Gates of the same type in the same layer are divided into groups of k which are evaluated together. Additionally, the number of times the output wires of each group are used as input wires in the subsequent layers is a multiple of k .

(Preprocessing Phase)

- Parties generate the required number of Beaver triples by invoking $\mathcal{F}_{\text{Triples}}$ (Functionality C.12) and linear transformation pairs using Protocol C.10. This comprises the preprocessing phase of the MPC protocol.

(Online Phase)

- Parties first divide gates of the same kind in every layer of the circuit into groups of size k .
- Following this, each party uses the VSS protocol from [12] to share their inputs to the circuit via degree- t Shamir sharing.
- Parties then invoke an instance of the ACS protocol to identify a subset of (at least) $n - t$ parties who successfully completed their sharing instances. A default value is considered as input for all the parties outside this set.
- Following this, the inputs to the subsequent layer have to be prepared.
 - For this, parties first apply the fan-out operation on groups of k wires using the linear transformation protocol. This generates the required number of copies of each input value shared by a party.
 - Subsequently, parties apply a permutation on the secrets, again considered in groups of size k . The guarantee offered by [19] is that upon permutation, the resultant shared values hold the following property. For each packed sharing $[z]_d$ that is used as an input to the subsequent layer, the values $z_1, \dots, z_k$ will be such that they lie at different positions in their respective degree- t sharing obtained as an output of the permutations. This allows parties to locally obtain a packed degree- d sharing of some permutation of z . Finally parties apply permutation again to obtain $[z]_d$.
- The addition and multiplication gates are then evaluated in a topological order. After each layer of evaluation, inputs to the subsequent layer are prepared as discussed above.
- Finally, after evaluating all the computation gates, parties proceed to the output phase. Here, parties reconstruct the packed sharing $[z]_d$ corresponding to a packed output wire belonging to some party P_s towards it by sending their shares. P_s performs online error correction on the received shares to obtain its output.

Theorem 3.11. *Let $t < n/4$. Protocol Π_{MPC} (Protocol C.16) securely implements $\mathcal{F}_{\text{MPC}}$ (Functionality 3.10) and has a communication complexity of $\mathcal{O}(|C'|n + n^5)$ bits over point-to-point channels in the preprocessing phase and $\mathcal{O}(|C'| + Dn^5 + n^5)$ bits in the online phase to evaluate a circuit C with depth D .*

The complete protocol appears in Appendix C.4, and the proof and the communication complexity analysis are given in Appendix C.5.

References

1. Abraham, I., Asharov, G., Patil, S., Patra, A.: Detect, pack and batch: Perfectly-secure mpc with linear communication and constant expected time. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques. pp. 251–281. Springer (2023)
2. Abraham, I., Asharov, G., Patil, S., Patra, A.: Perfect asynchronous mpc with linear communication overhead. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques. pp. 280–309. Springer (2024)
3. Abraham, I., Asharov, G., Patra, A., Stern, G.: Perfectly secure asynchronous agreement on a core set in constant expected time. IACR Cryptol. ePrint Arch. p. 1130 (2023), <https://eprint.iacr.org/2023/1130>

4. Applebaum, B., Brakerski, Z., Tsabary, R.: Perfect secure computation in two rounds. In: Theory of Cryptography: 16th International Conference, TCC 2018, Panaji, India, November 11–14, 2018, Proceedings, Part I 16. pp. 152–174. Springer (2018)
5. Applebaum, B., Brakerski, Z., Tsabary, R.: Degree 2 is complete for the round-complexity of malicious mpc. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques. pp. 504–531. Springer (2019)
6. Applebaum, B., Kachlon, E., Patra, A.: The round complexity of perfect mpc with active security and optimal resiliency. In: Annual Symposium on Foundations of Computer Science (FOCS) (2020)
7. Beaver, D.: Efficient multiparty protocols using circuit randomization. In: Annual International Cryptology Conference (1991)
8. Beerliová-Trubíniová, Z., Hirt, M.: Perfectly-secure mpc with linear communication complexity. In: Proceedings of the 5th Conference on Theory of Cryptography. p. 213–230. TCC’08, Springer-Verlag, Berlin, Heidelberg (2008)
9. Beerliová-Trubíniová, Z., Hirt, M.: Perfectly-secure mpc with linear communication complexity. In: Theory of Cryptography Conference (2008)
10. Canetti, R.: Studies in secure multiparty computation and applications (1996), <https://www.wisdom.weizmann.ac.il/~oded/PSX/ran-phd.pdf>
11. Canetti, R.: Studies in secure multiparty computation and applications. Ph.D. thesis, Citeseer (1996)
12. Choudhury, A., Patra, A.: An efficient framework for unconditionally secure multiparty computation. IEEE Transactions on Information Theory (2016)
13. Damgård, I., Ishai, Y., Krøigaard, M.: Perfectly secure multiparty computation and the computational overhead of cryptography. In: Advances in Cryptology - EUROCRYPT 2010. Lecture Notes in Computer Science, vol. 6110, pp. 445–465. Springer (2010)
14. Damgård, I., Nielsen, J.B.: Scalable and unconditionally secure multiparty computation. In: Annual International Cryptology Conference (2007)
15. Damgård, I.B., Li, B., Schwartzbach, N.I.: More communication lower bounds for information-theoretic mpc. In: 2nd Conference on Information-Theoretic Cryptography (ITC 2021). Schloss-Dagstuhl-Leibniz Zentrum für Informatik (2021)
16. Escudero, D., Goyal, V., Polychroniadou, A., Song, Y.: Turbopack: honest majority mpc with constant online communication. In: Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security. pp. 951–964 (2022)
17. Fitzi, M., Gottesman, D., Hirt, M., Holenstein, T., Smith, A.: Detectable byzantine agreement secure against faulty majorities. In: Proceedings of the twenty-first annual symposium on Principles of distributed computing. pp. 118–126 (2002)
18. Goyal, V., Liu, Y., Song, Y.: Communication-efficient unconditional mpc with guaranteed output delivery. In: Annual International Cryptology Conference. pp. 85–114. Springer (2019)
19. Goyal, V., Polychroniadou, A., Song, Y.: Unconditional communication-efficient mpc via hall’s marriage theorem. In: Advances in Cryptology–CRYPTO 2021: 41st Annual International Cryptology Conference, CRYPTO 2021, Virtual Event, August 16–20, 2021, Proceedings, Part II 41. pp. 275–304. Springer (2021)
20. Song, Y., Ye, X.: Perfectly-secure mpc with constant online communication complexity. Cryptology ePrint Archive (2024)

SUPPLEMENTARY MATERIAL

A Proofs from the lower bounds section

Lemma A.1 (lemma 2.17 from the main text). *Let π be a synchronous protocol that perfectly realizes $\mathcal{F}_{\text{LOT}}^\ell$ from secure channels for $n = 4$ and $t = 1$ corruptions (supporting both malicious and semi-honest corruptions), let P_i be a party (for some $i \in \{1, \dots, 4\}$), and let ts be a possible transcript of π where $\text{abort}(\text{ts}) = \text{leak}(\text{ts}) = 0$. Then there exists a protocol π' for $n' = 3$ parties and $t' = 1$ semi-honest corruptions that also perfectly realizes $\mathcal{F}_{\text{LOT}}^\ell$, such that there exists a party $P_{i'}$ of π' (for some $i' \in \{1, 2, 3\}$) and a possible transcript ts' of π' , such that $\text{abort}(\text{ts}') = \text{leak}(\text{ts}') = 0$ and $\text{TC}_{P_{i'}}(\text{ts}') \leq \text{TC}_{P_i}(\text{ts})$.*

Proof. Since parties P_3 and P_4 are symmetric in $\mathcal{F}_{\text{LOT}}^\ell$, we can assume without loss of generality that $i \leq 3$, and set $i' = i$.

Protocol. Define π' by having $\{P'_1, P'_2, P'_3, P'_{\text{audit}}\}$ run their corresponding parties $\{P_1, P_2, P_3, P_{\text{audit}}\}$ in π , while replacing P_4 with a fake, consisting of uniformly random messages. More precisely, every time some P_j (for $j \in \{1, 2, 3\}$) would send a message to P_4 , have P'_j send that message to P'_{audit} instead. And every time P_j would receive a message of length N from P_4 (according to the transcript ts), have P'_j sample the message m as $m \leftarrow \{0, 1\}^N$ instead, and send m to P'_{audit} . Let P'_2 abort if its π counterpart P_j does, and output whatever its counterpart would output.

Leakage is defined by P'_{audit} , who must decide whether the protocol behaved well enough to guarantee privacy. P'_{audit} will run P_4^H in its head, which is an honest version of P_4 , based on a fresh random tape and the messages directed at P_4 that were sent by the other parties P'_j . If in some round P_4^H sends a message m^H to P_j , while P'_j told P_{audit} that it sampled $m \neq m^H$ for this message, then P_{audit} will output 1 to declare this session leaky. If not, then also collect the messages P_4^H sends to P_{audit} , and run P_{audit} on these together with the P_{audit} messages sent by the other parties P'_j . P'_{audit} will declare leakage by outputting 1 if P_{audit} does, and otherwise it will output 0.

Simulation. We now define the simulator $\text{SIM}_{\pi'}$ for π' in terms of the simulator SIM_π for π . The party P_4 emulated by π' could be viewed as a maliciously corrupt party, so if none of P'_1, P'_2, P'_3 are corrupted then $\text{SIM}_{\pi'}$ can just emulate SIM_π with P_4 maliciously corrupted.¹³ We will now focus on the more difficult case where there is a real corrupted party: some P'_s has been semi-honest corrupted. Then it seems like we might have two corrupt parties. The trick is that either the fake P_4 behaves honestly, or P'_{audit} declares leakage and we can trivially simulate based on the protocol inputs.

Notice that since $\text{SIM}_{\pi'}$ is the simulator of a semi-honest protocol with only a single input round, it is almost non-interactive. That is, the simulator is given the inputs and outputs x'_s of P'_s ((m_0, m_1) if $s = 1$, (b, m_b) if $s = 2$, or empty if $s = 3$), sends $l \in \{0, 1\}$ to $\mathcal{F}_{\text{LOT}}^\ell$ to possibly request leakage, receives the leakage L , then outputs a decision $a \in \{0, 1\}$ of whether to abort and a view for the adversary view_A .¹⁴ The only interaction comes from the choice of whether to leak, so if hypothetically the simulator always knew the full input leakage L from the start, it would become just a distribution parameterized by L .¹⁵

Accordingly, we will define $\text{SIM}_{\pi'}$ in terms of distributions $\mathcal{D}_{\pi', L}^R$ and $\mathcal{D}_{\pi, L}^I$, with $\mathcal{D}_{\pi', L}^R$ sampling from the real execution of π' , and $\mathcal{D}_{\pi, L}^I$ sampling from the ideal execution of π using SIM_π (in case the fake P_4 happens to behave honestly). Here, $\mathcal{D}_{\pi', L}^R$ always samples an output, while $\mathcal{D}_{\pi, L}^I$ has some probability $1 - p(L)$ of failure, where it outputs $\perp$, instead of (l, a, view_A) for some leakage and abort request bits (l, a) and adversary view view_A . $\mathcal{D}_{\pi', L}^R$ consists of just running the real protocol π' on the inputs given by L , and collecting everything the simulator $\text{SIM}_{\pi'}$ needs to output: the message lengths, the view of P'_s , and whether leakage or abort occurs.

We will write $\mathcal{D}_{\pi, L}^I$ in terms of a simulator SIM_{P_4} that attempts to simulate the honest P_4 case using the original simulator SIM_π . SIM_{P_4} might or might not request leakage, and might abort by returning $\perp$ instead of a pair (l, a, view_A) . $\mathcal{D}_{\pi, L}^I$ uses L to emulate the interaction between SIM_{P_4} and $\mathcal{F}_{\text{LOT}}^\ell$, and

¹³ With one modification: it must request leakage from $\mathcal{F}_{\text{LOT}}^\ell$ more often than SIM_π does — whenever P_4 's messages disagree with the messages of a honest party P_4^H that SIM_π runs in its head.

¹⁴ This view view_A consists of the view of the semi-honest party P'_s , together with the lengths of all messages transferred over the secure channels.

¹⁵ Note that x'_s can be computed from L .

outputs what SIM_{P_4} outputs. SIM_{P_4} runs SIM_π for semi-honest P_s with x'_s as the inputs and outputs of P_s , forwards leakage requests to the functionality, then maps the resulting $\overline{\text{view}}_A$ to the corresponding view_A of π' . If in any round $\overline{\text{view}}_A$ and ts disagree over the length of message some party P_j receives from P_4 , we have found that the honest simulation has failed: have SIM_{P_4} output $\perp$. Otherwise, define the randomness that P'_s pretends are messages received from P_4 according to the $P_4 \rightarrow P_s$ messages in $\overline{\text{view}}_A$. The other messages of $\overline{\text{view}}_A$ can be directly copied into view_A . Also copy the leakage and abort decision made by SIM_π . This defines the output of SIM_{P_4} .

Now, we are ready to define the simulator $\text{SIM}_{\pi'}$ in terms of these distributions. Let $M = \text{SC}_{P_4}(\text{ts})$ be the total length of all messages sent by P_4 in ts . $\text{SIM}_{\pi'}$ starts by sampling a bit x , where $x = 0$ with probability 2^{-M} , and $x = 1$ otherwise. If $x = 0$, run SIM_{P_4} , but don't send $l = 0$ to $\mathcal{F}_{\text{LOT}}^\ell$ (yet) if SIM_{P_4} does not request leakage. If SIM_{P_4} does not output $\perp$ and leakage was not requested, then actually send $l = 0$ to $\mathcal{F}_{\text{LOT}}^\ell$. In any case, if SIM_{P_4} returns (l, a, view_A) , then $\text{SIM}_{\pi'}$ outputs (l, a, view_A) .

Alternatively, if $x = 1$ or SIM_{P_4} returned $\perp$, request leakage by sending $l = 1$ to $\mathcal{F}_{\text{LOT}}^\ell$ if it was not already requested, then simulate according to a distribution $\mathcal{D}_{\pi',L}^I$ which is defined by subtraction between $\mathcal{D}_{\pi',L}^R$ and $\mathcal{D}_{\pi,L}^I$. The idea is that the $x = 0$ case biased the marginal distribution for P_4 's messages towards honest messages, so when $x = 1$ we will bias in the opposite direction, to cancel it out. Let $\mathcal{D}_{\pi',L}^I = \frac{\mathcal{D}_{\pi',L}^R - 2^{-M} \mathcal{D}_{\pi,L}^I}{1 - 2^{-M} p(L)}$, meaning that the probability of non-failure output is proportional to the difference between the probabilities given by distributions $\mathcal{D}_{\pi',L}^R$ and $2^{-M} \mathcal{D}_{\pi,L}^I$, renormalized so that the probabilities defining $\mathcal{D}_{\pi',L}^I$ sum to 1. Then $\text{SIM}_{\pi'}$ will take a sample from $\mathcal{D}_{\pi',L}^I$, and output it.

Non-negative probabilities. For this simulation to be well defined, we need the subtraction to always output a non-negative probability. In fact, for the indistinguishability argument we will need that the subtraction always outputs 0 whenever $l = 0$.

$$\mathcal{D}_{\pi',L}^R(0, a, \text{view}_A) = 2^{-M} \mathcal{D}_{\pi,L}^I(0, a, \text{view}_A) \quad (2)$$

$$\mathcal{D}_{\pi',L}^R(1, a, \text{view}_A) \geq 2^{-M} \mathcal{D}_{\pi,L}^I(1, a, \text{view}_A) \quad (3)$$

Let $\mathcal{D}_{\pi,L}^R$ be the distribution consisting of executing the real world for π with P_s semi-honest, outputting failure $\perp$ if P_4 's communication pattern does not match ts , and otherwise mapping the resulting $\overline{\text{view}}_A$ to the corresponding view_A of π' . That is, following the same steps on the real world of π in exactly the same way as $\mathcal{D}_{\pi,L}^I$ was defined using the ideal world of π . By perfect security of π , we have $\mathcal{D}_{\pi,L}^I(l, a, \text{view}_A) = \mathcal{D}_{\pi,L}^R(l, a, \text{view}_A)$. During any execution of π that could produce (l, a, view_A) , the communication pattern of P_4 exactly matches ts . If during an execution of π' on the same inputs, parties $\{P'_1, P'_2, P'_3\}$ and the honest P_4^H run by P_{audit} make have the same random tapes as in π , and the randomness used to sample P_4 's messages equals the plaintext messages in the execution of π (i.e. the messages which would be sent by P_4^H), then this execution will lead to the exact same (l, a, view_A) . The probability of a fake P_4 message of length N being sampled to match the corresponding N -bit message in the execution of π is exactly 2^{-N} , and so the probability of all fake P_4 messages matching is 2^{-M} . Therefore, every possible execution of π with probability ϵ of occurring, has a corresponding execution of π' with probability at least $2^{-M} \epsilon$ of occurring, so $2^{-M} \mathcal{D}_{\pi,L}^R(l, a, \text{view}_A) \leq \mathcal{D}_{\pi',L}^R(l, a, \text{view}_A)$. Finally, the $l = 0$ case gives equality because there is no other way than the above for $\mathcal{D}_{\pi',L}^R(0, a, \text{view}_A)$ to produce a matching $(0, a, \text{view}_A)$: if any message sent by the fake P_4 does not match the honest P_4^H run by P_{audit} , then P_{audit} outputs $l = 1$.

Indistinguishability. We now show that if this simulation is well-defined, then it is perfectly secure. We divide the simulation distributions into cases based on $x \in \{0, 1\}$ and $l \in \{0, 1\}$. Note that $l = 0$ is only possible when $x = 0$. When $x = 0$, the simulator runs SIM_{P_4} and interacts with $\mathcal{F}_{\text{LOT}}^\ell$; by definition the distribution output given by this interaction is exactly $\mathcal{D}_{\pi,L}^I$. Therefore, for $l = 0$ the real and ideal probability distributions are equal:

$$\Pr[x = 0] \cdot \mathcal{D}_{\pi,L}^I(0, a, \text{view}_A) = 2^{-M} \mathcal{D}_{\pi,L}^I(0, a, \text{view}_A) = \mathcal{D}_{\pi',L}^R(0, a, \text{view}_A),$$

using (2).

Next we consider $l = 1$, and dividing into cases based $x \in \{0, 1\}$ and whether SIM_{P_4} outputs $\perp$. When $x = 1$ or SIM_{P_4} outputs $\perp$ then $\text{SIM}_{\pi'}$ uses $\mathcal{D}_{\pi',L}^I$; by (2) this always produces a sample (l', a, view_A)

where $l' = 1$, as the subtracted probabilities exactly cancel for $l' = 0$. Then the probability of $\text{SIM}_{\pi'}$ sampling (l, a, view_A) is exactly

$$\begin{aligned}
& \Pr[x = 0] \cdot \mathcal{D}_{\pi, L}^I(1, a, \text{view}_A) + (\Pr[x = 1] + \Pr[x = 0] \cdot \mathcal{D}_{\pi, L}^I(\perp)) \cdot \mathcal{D}_{\pi', L}^I(1, a, \text{view}_A) \\
&= 2^{-M} \mathcal{D}_{\pi, L}^I(1, a, \text{view}_A) + (1 - 2^{-M} + 2^{-M}(1 - p(L))) \mathcal{D}_{\pi', L}^I(1, a, \text{view}_A) \\
&= 2^{-M} \mathcal{D}_{\pi, L}^I(1, a, \text{view}_A) + (1 - 2^{-M} p(L)) \mathcal{D}_{\pi', L}^I(1, a, \text{view}_A) \\
&= 2^{-M} \mathcal{D}_{\pi, L}^I(1, a, \text{view}_A) + \mathcal{D}_{\pi', L}^R(1, a, \text{view}_A) - 2^{-M} \mathcal{D}_{\pi, L}^I(1, a, \text{view}_A) \\
&= \mathcal{D}_{\pi', L}^R(1, a, \text{view}_A),
\end{aligned}$$

so the simulation is identically distributed to the real world.

Transcripts. Finally, since it is possible for ts to be produced by SIM_{π} , it is also possible for it to be produced when SIM_{P_4} runs SIM_{π} internally. In this case, the message pattern in ts will not cause SIM_{P_4} to output failure, so $\text{SIM}_{\pi'}$ will produce a corresponding transcript ts' , with $\text{abort}(\text{ts}') = \text{abort}(\text{ts}) = 0$ and $\text{leak}(\text{ts}') = \text{leak}(\text{ts}) = 0$. $\text{SIM}_{\pi'}$ runs SIM_{P_4} with probability 2^{-M} , so if there is probability ϵ of SIM_{π} producing ts then there is probability $2^{-M}\epsilon$ of $\text{SIM}_{\pi'}$ producing ts' . $\square$

Lemma A.2 (Lemma 2.18 from the main text). *Let π be a synchronous protocol that perfectly realizes $\mathcal{F}_{\text{LOT}}^\ell$ from secure channels for $n = 3$ and $t = 1$ semi-honest corruptions, and let $i \neq i' \in \{1, \dots, 3\}$ and $\ell' \geq 0$. Then there exists a protocol π' that perfectly realizes $\mathcal{F}_{\text{LKA}}^\ell$ between parties Alice and Bob, using an authenticated insecure channel $\mathcal{F}_{\text{AC}}$ and ℓ' bits of correlated randomness $\mathcal{F}_{\text{PSK}}^{\ell'}$. If it is possible that π neither leaks nor aborts while the channel between P_i and $P_{i'}$ has total communication at most ℓ' , then it is also possible that π' neither leaks nor aborts. Additionally, if there is a possible transcript ts of π where $\text{TC}_{P, P'}(\text{ts}) \leq \ell'$ and $\text{abort}(\text{ts}) = \text{leak}(\text{ts}) = 0$, there exists a possible transcript ts' of π' such that $\text{abort}(\text{ts}') = \text{leak}(\text{ts}') = 0$.*

Proof. Assume without loss of generality that $i < i'$. Let the parties Alice and Bob in π' emulate parties in π , with Alice emulating P_1 and Bob emulating P_2 . If $\{i, i'\} = \{2, 3\}$, let Alice additionally emulate P_3 , and otherwise let Bob emulate P_3 ; consequentially communication between P_i and $P_{i'}$ will always go between Alice and Bob. Use ℓ' bits of correlated randomness to define a one-time pad to encrypt the messages between P_i and $P_{i'}$; if the one-time pad runs out, stop communicating, and have Bob abort. Alice samples a random key $k \leftarrow \{0, 1\}^\ell$. She will input $m_0 = m_1 = k$ to π as P_1 , and Bob will sample a random $b \leftarrow \{0, 1\}$ to input as P_2 . When π completes, Alice outputs k , and if Bob does not abort then he receives $m_b = k$ from π and outputs it.

To prove security of π' , we will define the simulator $\text{SIM}_{\pi'}$ for π' in terms of the simulator SIM_{π} for π . We now divide into two cases:

If $\{i, i'\} = \{1, 2\}$: To simulate π' , run SIM_{π} with semi-honest corrupt P_3 ; the view of P_3 then contains the communication between P_1 and P_3 . In the simulation of π' , this will be the unencrypted communication between Alice and Bob, corresponding to messages sent between their emulations of P_1 and P_3 . Simulate all encrypted traffic between Alice and Bob corresponding to messages between P_1 and P_2 uniformly at random (since they are encrypted with a one-time pad in the real world), with lengths given by the message lengths output by SIM_{π} . If a message is reached where more than ℓ' bits of communication would be sent between P_1 and P_2 (so the one-time pad would be exhausted), abort. Additionally, abort if SIM_{π} says to. If SIM_{π} requests leakage, sample a random bit b and a random key k^* , and program k^* into $\mathcal{F}_{\text{LKA}}^\ell$ as the leakage by sending $(1, k^*)$, and then respond to SIM_{π} with $L = (k^*, k^*, b)$. By inspection, the simulator $\text{SIM}_{\pi'}$ is perfect if SIM_{π} is perfect, and if it is possible for SIM_{π} to not request leakage and not abort then this is also possible for $\text{SIM}_{\pi'}$.

If $i' = 3$: Sample a random bit $b \in \{0, 1\}$ and key $k^* \in \{0, 1\}^\ell$, and run SIM_{π} with corrupt P_2 with b as P_2 's input and k^* as its output. This simulation will output a view for P_2 , which will contain the communication between P_1 and P_2 . Simulate the communication between P_i and P_3 uniformly at random, since it is encrypted with a one-time pad in the real world. Use these two sets of messages to simulate the corresponding traffic between Alice and Bob. Abort if the simulator for π says to, or if its simulation of the channel between P_i and P_3 says that more than ℓ' bits of communication occurred.

If SIM_π requests leakage, program k^* into $\mathcal{F}_{\text{LKA}}^\ell$ as the leakage by sending $(1, k^*)$, and then respond to SIM_π with $L = (k^*, k^*, b)$.

We now give two hybrid arguments to show that this simulation is identically distributed to the real world. In the first hybrid argument, we argue the case where leakage does not occur — i.e., if leakage does occur then we set the distinguisher's view to $\perp$ instead of allowing it to see the environment. In the second hybrid argument, we will instead argue the case where leakage does occur. Together, these prove that the simulation is identically distributed to the real world. The one subtlety is that in both arguments we must be careful to not change whether leakage occurs. This will hold because of P_{audit} guaranteeing that l is well-defined in the real world, and must agree with l in the ideal world.

If leakage does not occur ($l = 0$):

1. This is the real world, where Alice and Bob each run their corresponding (sets of) parties in π .
2. Use the one-time pad to change the adversary's view of communication between P_i and P_3 (which are part of the communication between Alice and Bob) to be uniformly random, but with the correct length. Treat the adversary's view of the messages between P_1 and P_2 as coming from the state of a semi-honest corrupted P_2 .
3. Using perfect security of π , change to the $\mathcal{F}_{\text{LOT}}^\ell$ ideal world, where the (emulated) communication between P_1 and P_2 comes from the simulator SIM_π .
4. Change P_1 's input to (m_0, m_1) where $m_b = k$ and $m_{1-b} = k^*$. This cannot change whether leakage occurs, as SIM_π is given only the view of P_2 when deciding whether to leak, and P_2 's view is still (b, k) .
5. Using perfect security of π , change π back to the real world.
6. Corrupt P_1 instead of P_2 . Since P_1 and P_2 have the same view of the authenticated channel between them, we can still get the emulated communication between from the corrupted party's view.
7. Using perfect security of π , change to the $\mathcal{F}_{\text{LOT}}^\ell$ ideal world again.
8. Have Bob always output k when he does not abort, irrespective of what $\mathcal{F}_{\text{LOT}}^\ell$ outputs. Since $m_b = k$, $\mathcal{F}_{\text{LOT}}^\ell$ would always output k anyway.
9. Swap the OT messages so that $m_{1-b} = k$ and $m_b = k^*$. This is identically distributed to flipping Bob's input bit, which is uniformly random and independent of the views of SIM_π and the environment. Therefore, this cannot change whether leakage occurs.
10. Change π back to the real world.
11. Corrupt P_2 instead of P_1 .
12. Change π to the $\mathcal{F}_{\text{LOT}}^\ell$ ideal world again.
13. View the sampling of b and k^* , and the execution of SIM_π , as being part of a simulator $\text{SIM}_{\pi'}$. View the sampling of k , output by both parties, as being part of a functionality $\mathcal{F}_{\text{LKA}}^\ell$. This is now identical to the ideal world for π' (when leakage does not occur).

If leakage does occur ($l = 1$):

1. This is the real world, where Alice and Bob each run their corresponding (sets of) parties in π .
2. Use the one-time pad to change the adversary's view of communication between P_i and P_3 (which are part of the communication between Alice and Bob) to be uniformly random, but with the correct length. Treat the adversary's view of the messages between P_1 and P_2 as coming from the state of a semi-honest corrupted P_2 .
3. Using perfect security of π , change to the $\mathcal{F}_{\text{LOT}}^\ell$ ideal world, where the (emulated) communication between P_1 and P_2 comes from the simulator SIM_π .
4. Rename k to k^* . View the parties outputting k^* as running $\mathcal{F}_{\text{LKA}}^\ell$ with k^* programmed in using the leakage. This is now identical to the ideal world for π' (when leakage does not occur).

Additionally, if there is a possible transcript ts of π where $\text{TC}_{P, P'}(\text{ts}) \leq \ell'$ and $\text{abort}(\text{ts}) = \text{leak}(\text{ts}) = 0$, there exists a possible transcript ts' of π' such that $\text{abort}(\text{ts}') = \text{leak}(\text{ts}') = 0$. Finally, we need to show that if there is a possible transcript ts of π where $\text{TC}_{P, P'}(\text{ts}) \leq \ell'$ and $\text{abort}(\text{ts}) = \text{leak}(\text{ts}) = 0$, then it is possible for $\text{SIM}_{\pi'}$ to not request leakage or abort from $\mathcal{F}_{\text{LKA}}^\ell$. By perfect security of π , it must be possible for ts to be produced by some SIM_π execution, for some inputs to π , when P_2 is corrupted. Let b and m_b be a view of P_2 such that it's possible for SIM_π to produce ts when given this view. Then there is probability $2^{-\ell-1}$ that $\text{SIM}_{\pi'}$ will choose this b and $k^* = m_b$. $\text{SIM}_{\pi'}$ then leaks only if SIM_π does the same, and aborts only if SIM_π aborts or the total communication between P_i and P_3 is more than ℓ' . Since it is possible for SIM_π to produce ts in this case, it is possible for $\text{SIM}_{\pi'}$ to simultaneously not leak and not abort. $\square$

B VSS and MPC for Linear Functions when $n > 2t$

The lower bound we showed earlier rules out abort secure MPC in general when we only have $n > 2t$. However the proof uses specifically computation of a non-linear function. In this section, we show that VSS and secure computation of linear functions can be done with perfect abort security assuming only $n > 2t$.

We first need to discuss in some detail what VSS with perfect abort security should actually achieve. At first sight, such a VSS may seem trivial to achieve: the dealer can just send Shamir shares of her secret to all players using a random polynomial of degree at most t as usual. At reconstruction time, either all players report shares that are on one polynomial f of degree at most t , in which case $f(0)$ is returned; and otherwise the protocol aborts.

This works, at least to some extent: for an honest dealer, privacy is obvious and since the honest players determine the polynomial used, either the correct secret is reconstructed or we abort. For a dishonest dealer, either all honest players receive points on a polynomial of degree at most t or they do not. In the former case, the secret is well defined and may be reconstructed. In the latter case, no secret is defined, but one could decide that this is fine since reconstruction will always fail, and aborting is ok in this model.

However, such a VSS would be problematic when used for MPC: suppose we construct a protocol for adding two private values in the standard way where the two values are VSS'ed, players add the shares they get, and the result is reconstructed. Now, a corrupt dealer can do the following: she shares values s, s' correctly, except that she adds an error e to player P_i 's share of s and adds $-e$ to the P_i 's share of s' . Neither of these sharings are consistent with a polynomial of the correct degree, so it is not clear how a simulator could extract meaningful input values. Nevertheless, once the shares are added, the error cancels out and the protocol may return $s + s'$ without aborting, which would make the simulation fail.

To avoid this problem, we need to insist that, while the sharing phase may abort, if it does not, a well-defined secret must exist. One way to capture this is to require an abort secure VSS to be a perfectly secure implementation of the following functionality.

Functionality B.1: VSSwAbort - VSS with abort

Sharing Phase When invoked by all honest players, send “VSS started” to the adversary.

If the dealer is corrupted, the adversary may return a value s or “abort”. In the first case, send “success” to all players and store s . Otherwise, send $\perp$ to all players and stop.

If the dealer is honest, get input s from the dealer. The adversary may return “OK” or “abort”. In the first case, send “success” to all players and store s . Otherwise, send $\perp$ to all players and stop.

Reconstruction On input “reconstruct” from all honest players, send s to the adversary. If the adversary returns “OK”, send s to all players. Otherwise, send $\perp$ to all players and stop.

Using a variant of the well-known bi-variate polynomial technique, this functionality is easy to implement. For simplicity we will use symmetric polynomials $f(x, y)$ over some finite field $\mathbb{F}_p$, where the coefficient of $x^a y^b$ is the same as that of $x^b y^a$. This implies that $f(i, j) = f(j, i)$. We suggest the following protocol:

Protocol B.2: VSS Protocol $\pi_{\text{VSSwAbort}}$

Sharing Phase .

1. To share a value s , the dealer samples a random bivariate polynomial $f(X, Y)$ of degree at most t in both variables subject to $f(0, 0) = s$, and sends the polynomial $f_i(X) = f(X, i)$ to P_i , for each player P_i .
2. Each P_i sends privately $f_i(j) = f(j, i)$ to each player P_j .
3. Each player P_j verifies that the value she received for each P_i equals $f_j(i) = f(i, j)$. If a mismatch is found, she broadcasts “abort”, else she broadcasts “OK”.
4. If all players broadcasted “OK”, each player P_i outputs $f_i(0)$ as her share, otherwise, the protocol aborts.

Reconstruction .

1. All players broadcast their share.

2. If all shares broadcast are on one polynomial $g(X)$ of degree at most t , then players output $g(0)$. Otherwise, the protocol aborts.
-

Theorem B.3. *Assuming $t < n/2$ maliciously corrupted players, $\pi_{\text{VSSwAbort}}$ is a perfectly secure implementation of $F_{\text{VSSwAbort}}$*

Proof. We only sketch the proof, as it is straightforward and resembles many proofs from the literature.

For an honest dealer, in the sharing phase the simulator emulates the honest dealer using secret $s = 0$. This is a perfect simulation as the adversary does not see more than t shares in this phase. If the corrupt players send incorrect values in round 2, the simulator sends “abort” to the functionality, else send “OK”. In the reconstruction phase, the simulator is given s , and patches the set of shares it holds for honest players, so they are consistent with s and the shares held by corrupt players. It sends the resulting shares to the corrupt players. If a corrupt player sends an incorrect share, send “abort” to the functionality, otherwise send “OK”.

For a corrupt dealer, the simulator plays for the honest players following the protocol in the sharing phase. If the protocol aborts, send “abort” to the functionality, else send “OK”. Note that if the protocol did not abort, all pairs of honest players P_i, P_j hold polynomials $f_i(X), f_j(X)$ satisfying $f_i(j) = f_j(i)$. For simplicity, assume that the honest players are $P_1, \dots, P_{t+1}$ and let $\gamma_1, \dots, \gamma_{t+1}$ be Lagrange coefficient for reconstructing the value in 0 of a degree t polynomial from its values in $1, \dots, t+1$. Let $g(X) = \sum_{j=1}^{t+1} \gamma_j f_j(X)$. Clearly, $g(X)$ is of degree at most t , and we have that players will output shares consistent with $g(X)$, namely for an honest player P_i we have

$$g(i) = \sum_{j=1}^{t+1} \gamma_j f_j(i) = \sum_{j=1}^{t+1} \gamma_j f_i(j) = f_i(0)$$

The simulator can therefore interpolate $g(X)$, set $s = g(0)$ and send s to the functionality. For reconstruction, the simulator sends the shares it has for the honest players. If a corrupt player sends an incorrect share (one that is not consistent with the polynomial g), send “abort” to the functionality, else send “OK”. $\square$

We sketch the (quite obvious) approach to doing linear secure computation based on the above VSS: Two sets of shares (of a and b) created via $\pi_{\text{VSSwAbort}}$ can be added locally and will produce shares of $a + b$, where all honest players have consistent shares. This implies that reconstruction from these shares will result in $a + b$ or abort. Something similar can be observed for multiplication by a constant. Hence, to securely apply linear function ϕ to a set of values $a_1, \dots, a_m$, we ask the holder of each a_i to deal a VSS where a_i is the secret. Then all players apply ϕ to their shares, and reconstruct the result. It follows from the observations we just made that the result must be $\phi(a_1, \dots, a_m)$ or abort, and that we never abort if all players are honest. Privacy follows immediately from privacy of the VSS and the fact that, as long as at least one input is held by an honest player, the output sharing is random with the only constraint that it determines $\phi(a_1, \dots, a_m)$.

To summarize, we have the following:

Theorem B.4. *For any function ϕ that is linear over a finite field, there exists a protocol that computes ϕ with perfect abort security, provided $t < n/2$.*

C Asynchronous Perfect MPC with Constant Online Overhead

C.1 More Preliminaries

Symmetric Bivariate Polynomials A degree (ℓ, ℓ) symmetric bivariate polynomial over $\mathbb{F}$ is of the form $S(x, y) = \sum_{i=0}^{\ell} \sum_{j=0}^{\ell} c_{ij} x^i y^j$ where $c_{ij} \in \mathbb{F}$ and $c_{ij} = c_{ji}$ holds for all $i, j \in \{0, \dots, \ell\}$. This implies $S(i, j) = S(j, i)$ holds for every i, j . Moreover, $S(x, i) = S(i, y)$ is true for each $i \in \mathbb{F}$. Our protocol uses (t, t) -symmetric bivariate polynomials.

Agreement on a Core Set In our asynchronous protocols, we also make use of an existing primitive which is called the agreement on a core set or asynchronous common subset, commonly known as ACS. The functionality for the same, $\mathcal{F}_{\text{ACS}}$, as used in our work appears below. We use the protocol from [3] which has a cost of $\mathcal{O}(n^5)$ field elements of communication with a constant expected run time.

Functionality C.1: $\mathcal{F}_{\text{ACS}}$ - Agreement on a Core Set

The functionality is parameterized by a set of corrupted parties $\mathcal{C} \subseteq [n]$. Initialize sets $S_i \leftarrow \emptyset$ for every $i \in [n]$ and $S \leftarrow \perp$. In addition, initialize $\text{returned} \leftarrow 0$.

1. (**record**, i, s): Upon receiving this command from party i , add the index s to S_i . Forwards (**record**, i, s) to the adversary. If $|S_i| \geq n - t$ then set i as **ready**. If $n - t$ honest parties are **ready**, then set S to be the set of all indices $s \in [n]$ such that there exists some $\ell \notin \mathcal{C}$ for which (**record**, ℓ, s) was sent.
 2. (**set**, S'): Upon receiving this command from the adversary, check that $S' \subseteq [n]$ and that $|S'| \geq n - t$. Moreover, check that for every $s \in S'$, there exists some $\ell \notin \mathcal{C}$ for which $s \in S_\ell$ (i.e., P_ℓ has submitted (**record**, ℓ, s)). If all those conditions hold, and $\text{returned} = 0$, then store $S \leftarrow S'$.
 3. **receiveS**($\cdot$): Upon receiving this command from some party i , if $S \neq \perp$, set $\text{returned} \leftarrow 1$. Return S .
-

C.2 Preprocessing Phase

Reconstruction using Online Error Correction Online error correction was a procedure proposed by Canetti [11] for reconstructing shared polynomials in an asynchronous network, and it involves applying the Reed Solomon error correction repeatedly in an online manner. Particularly, suppose that a degree d^* polynomial has to be reconstructed towards a party P^* . For this, all the parties disclose their shares to P^* who tries to recover the polynomial as follows. It waits for $d^* + t + 1$ shares, all of which lie on the same degree d^* polynomial. If P^* obtains such a polynomial then it is guaranteed to be correct since it must agree with the shares of at least $d^* + 1$ honest parties. Given that t corrupt parties may never communicate, this procedure is successful and terminates only when $n - t > d^* + t$ or equivalently, $n - 2t > d^*$. In our work, the highest degree of polynomials we operate with is $d + k - 1 = 2t$ which is within the required bounds. The protocol for reconstruction appears below. Throughout this work, when we refer to a party reconstructing a value, we implicitly assume that the following procedure using online error correction is employed.

Protocol C.2: Reconstruction Protocol

Common input: The description of a field $\mathbb{F}$, n non-zero distinct elements $1, \dots, n$, identity of a party P^* .

Input: Parties hold the univariate degree- d^* sharing $[v]_{d^*}$.

1. Each P_i sends its share $[v]_{d^*}^i$ to P^* and terminates.
 2. For $r = 0, \dots, t$:
 - (a) Upon receiving the first $d^* + t + 1 + r$ values, P^* looks for a codeword of a polynomial of degree- d^* with a distance of at most r from the values it received. If there is no such unique codeword, P^* proceeds to the next iteration. Otherwise, it sets $p_r(x)$ to be the unique RS reconstruction.
 - (b) If $p_r(i) = [v]_{d^*}^i$ holds for at least $d^* + t + 1$ parties whose shares were considered during RS reconstruction, then P^* outputs $p_r(0)$ and terminates. Otherwise, it proceeds to the next iteration.
-

Lemma C.3. *Protocol C.2 has a communication complexity of $\mathcal{O}(n)$ elements over point-to-point channels to reconstruct a secret privately to a party.*

Preparing Random Sharings Our protocol requires parties to prepare the following kinds of degree- t Shamir sharings of random values.

- Random degree- t Shamir sharings of the kind $[r|i]_t$ for each $i \in \{1, \dots, k\}$ such that r is not known to any party. That is, each P_u is required to hold $f(u)$ for some degree- t polynomial $f(x)$ such that $f(-i) = r$.

- Random pairs of degree- t Shamir sharings of the type $[r|i]_t, [r|j]_t$ for every $i, j \in \{1, \dots, k\}$ where r is not known to any party. In this case, each P_u is required to hold $f(u), g(u)$ for some degree- t polynomials $f(x), g(x)$ such that $f(-i) = g(-j) = r$. We often refer to these as random (i, j) sharings.

Following the approach of [20], we use the ideas in [9] to prepare p such random sharings at a cost of $\mathcal{O}(np + n^5)$. Below, we describe the protocols for computing these types of random sharings.

In the following protocol, every party invokes the VSS protocol to share random values via degree- t Shamir sharing. Note, however, that parties cannot afford to wait for all parties' sharing instances to terminate. Doing so might result in an endless wait since the corrupt parties may not initiate an instance of sharing. Hence, parties must agree on a common set **Com** of at least $n-t$ parties whose sharing instances will eventually terminate for all the parties. For this, parties execute an instance of ACS to decide on the set of common parties whose shared values will be used for further computation. Following this, parties extract shares of truly random values unknown to any party using the hyper-invertible matrix technique [9] on the values shared by parties in **Com**. Protocol C.4 has a communication complexity of $\mathcal{O}(n^5)$ elements over point-to-point channels to share $\mathcal{O}(n^2)$ random elements via degree- t Shamir sharing. Note that the cost of broadcast within the VSS, and the ACS invocation can be amortized over many instances to obtain $\mathcal{O}(n)$ communication cost per sharing. We refer to this variant as the batched execution or batching mode of random sharing.

Protocol C.4: Random Sharing Protocol

- **The protocol:**
 1. Each P_s invokes an instance of the packed VSS protocol of [12] with a degree $(t+k, t)$ -degree bivariate polynomial to share $[r_j^s|j]_t$ for each $j \in \{1, \dots, k\}$.
 2. Parties run an instance of the ACS protocol to identify a common set of at least $n-t$ parties that completed the prior step. Let **Com** denote this set and w.l.o.g let us assume $\text{Com} = \{P_1, \dots, P_{n-t}\}$.
 3. Parties compute $([\tilde{r}_j^1|j]_t, \dots, [\tilde{r}_j^{n-2t}|j]_t) = M \cdot ([r_j^1|j]_t, [r_j^2|j]_t, \dots, [r_j^{n-t}|j]_t)$ for each $j \in \{1, \dots, k\}$ where M is a hyper-invertible matrix of size $(n-2t) \times (n-t)$.
-

Lemma C.5. *Protocol C.4 when run in the batching mode has a communication complexity of $\mathcal{O}(np+n^5)$ elements over point-to-point channels to share p random elements via degree- t Shamir sharing.*

For preparing the second type of random sharing, each party P_s first distributes pairs of degree- t Shamir sharings of the form $[r|i]_t, [r|j]_t$ such that r is known to P_s . Following this, parties use the hyper-invertible matrix technique from [9,20] to verify the validity of these sharings. That is, parties ensure that the same value r is indeed shared via some degree- t polynomials $f(x), g(x)$ such that $f(-i) = g(-j) = r$. If the verification for some party P_s fails, then its shared values are not considered in the subsequent computation. Following this, parties extract truly random pairs of sharings from those shared by each party individually. To ensure this, while avoiding an endless wait, parties execute an instance of ACS protocol and identify a subset **Com** of at least $n-t$ parties that successfully shared their random values and passed the verification. The random sharings dealt by parties in **Com** are then used to generate sharings of pairs of random values similar to Protocol C.4. Protocol C.6 has a communication complexity of $\mathcal{O}(n^5)$ elements over point-to-point channels to share $\mathcal{O}(n^2)$ random elements via degree- t Shamir sharing. As in the prior case, the broadcast cost and the cost for ACS invocation can be amortized over many instances batched together. The protocol (without batching) appears below.

Protocol C.6: Random (i, j) Sharing Protocol

- **The protocol:**
 1. Each P_s invokes an instance of the packed VSS protocol of [12] to share $[r_m^s|i]_t, [r_m^s|j]_t$ for some $i, j \in \{1, \dots, k\}$ and $m \in \{1, \dots, n-2t\}$.
 2. Parties compute $([\tilde{r}_1^s|i]_t, \dots, [\tilde{r}_n^s|i]_t) = M \cdot ([r_1^s|i]_t, [r_2^s|i]_t, \dots, [r_{n-2t}^s|i]_t)$ where M is a hyper-invertible matrix of size $n \times (n-2t)$.
 3. Parties compute $([\hat{r}_1^s|j]_t, \dots, [\hat{r}_n^s|j]_t) = M \cdot ([r_1^s|j]_t, [r_2^s|j]_t, \dots, [r_{n-2t}^s|j]_t)$ where M is a hyper-invertible matrix of size $n \times (n-2t)$.

4. For each party P_s , parties reconstruct $[\tilde{r}_\ell^s|i]_t, [\hat{r}_\ell^s|j]_t$ to every P_ℓ .
5. P_ℓ broadcasts $\text{OK}(s, \ell)$ if $\tilde{r}_\ell^s = \hat{r}_\ell^s$ holds.
6. Parties accept P_s if at least $n - t$ parties broadcast OK and output the shares of $n - 3t$ pairs of arbitrary points. Let these be denoted by $\{([\tilde{r}_1^s|i]_t, [\tilde{r}_1^s|j]_t), \dots, ([\tilde{r}_{n-3t}^s|i]_t, [\tilde{r}_{n-3t}^s|j]_t)\}$
7. Parties run an instance of the ACS protocol to identify a common set of at least $n - t$ parties that successfully completed the sharing in prior steps. Let Com denote this set and w.l.o.g let us assume $\text{Com} = \{P_1, \dots, P_{n-t}\}$.
8. Parties compute

$$([\tilde{r}_u^1|i]_t, \dots, [\tilde{r}_u^{n-2t}|i]_t) = M \cdot ([r_u^1|i]_t, [r_u^2|i]_t, \dots, [r_u^{n-t}|i]_t)$$

and

$$([\hat{r}_u^1|j]_t, \dots, [\hat{r}_u^{n-2t}|j]_t) = M \cdot ([r_u^1|j]_t, [r_u^2|j]_t, \dots, [r_u^{n-t}|j]_t)$$

for each $u \in \{1, \dots, n - 3t\}$ where M is a hyper-invertible matrix of size $(n - 2t) \times (n - t)$.

Lemma C.7. *Protocol C.6 when run in the batching mode has a communication complexity of $\mathcal{O}(np + n^5)$ elements over point-to-point channels to share p pairs of (i, j) random elements via degree- t Shamir sharing.*

Transpose In [20], the generation of randomness for *different* linear transformations with the required communication complexity was reduced to performing the *same* operation of transpose on matrices of size $k \times k$ where each entry x_{ij} from the i th row and j th column in the matrix is shared via a degree- t polynomial $f_{ij}(x)$ such that $f_{ij}(-j) = x_{ij}$. We mostly follow their protocol for transposing, with some minor changes. At a high level, their protocol for transpose works as follows. Let the matrix to be transposed be denoted by X . For this, the preprocessing data required is the random (i, j) sharing discussed in Protocol C.6 for each $i, j \in \{1, \dots, k\}$. Note that the randomness itself can be viewed as a $k \times k$ matrix R with entries r_{ij} for each $i, j \in \{1, \dots, k\}$. Parties first locally compute the degree- d packed sharing of each row of the matrix X as well as R , say $\mathbf{x}_i$ and $\mathbf{r}_i$ respectively, where $i \in \{1, \dots, k\}$. Following this, parties locally use $\mathbf{r}_i$ to mask the row $\mathbf{x}_i$ and open all the masked rows towards a designated party known as P_{king} . This essentially allows P_{king} to learn the masked matrix $X + R$, and thus it can compute the transpose of this masked matrix locally. In their protocol, P_{king} finally shares a degree- $(k - 1)$ sharing of each row of the masked and transposed matrix. Using this and the existing degree- t sharing of r_{ij} values from the random (i, j) sharing, parties leverage the linearity of sharing to compute the shares of each entry of the transposed matrix. That is, P_{king} in effect performs degree reduction of a shared matrix among parties in order to perform the transpose operation, and the only communication in the protocol is to and from the P_{king} . Finally, to tackle the malicious behaviour of parties while reconstructing towards P_{king} , or of P_{king} sharing the incorrect values, their work makes use of the dispute control technique and identifies conflict pairs.

Translating the above protocol to the asynchronous setting has immediate problems, as discussed earlier. Specifically, we note that relying on a single designated party P_{king} can result in an endless wait. This is because it is impossible to differentiate between a party that is slow, and a party that is corrupt and not initiating any communication in an asynchronous network. Further, the dispute control approach requires parties to broadcast their view of the computation to identify a pair of parties which disagree such that at least one of them is corrupt. Once again, this is problematic in the asynchronous network. The corrupt parties may choose to not broadcast at this stage, potentially causing an endless wait. Owing to this, to ensure a protocol in the asynchronous network, we require a different mechanism. Here, we observe that the transpose protocol is only instantiated in the preprocessing phase, and this allows us to leverage the $\mathcal{O}(n)$ cost per gate that is allowed in this phase. Hence, instead of a designated P_{king} performing the reconstruction, we allow parties to perform a public reconstruction of the $X + R$ matrix using the broadcast of [3]. We ensure a cost of $\mathcal{O}(n)$ per gate by batching the public reconstruction of all such matrices in the preprocessing phase. We describe the protocol below.

Protocol C.8: Transpose Protocol

- **Input:** Parties hold $\{[x_{ij}|j]_t\}_{i,j=1}^k$ which can be viewed as a $k \times k$ matrix X . They also hold $\{[r_{ij}|j]_t, [r_{ji}|j]_t\}_{i,j=1}^k$ computed using Protocol C.6. These can be viewed as $k \times k$ matrices R, R^T .
 - **The protocol:**
 1. Parties compute $[\mathbf{x}_i]_d = \sum_{j=1}^k [e_j]_{k-1} * [x_{ij}|j]_t$ and $[\mathbf{r}_i]_d = \sum_{j=1}^k [e_j]_{k-1} * [r_{ij}|j]_t$ for each $i \in \{1, \dots, k\}$ locally.
 2. Parties compute the sharing of the i th row of $X + R$ as $[\mathbf{x}_i + \mathbf{r}_i]_d = [\mathbf{x}_i]_d + [\mathbf{r}_i]_d$ locally.
 3. Parties robustly and publicly reconstruct¹⁶ $\mathbf{x}_i + \mathbf{r}_i$ for each $i \in \{1, \dots, k\}$ and thus obtain the $k \times k$ matrix $X + R$. Parties locally obtain the transposed matrix $(X + R)^T$.
 4. Each party locally computes the default degree- $(k-1)$ sharing of the i th row $\mathbf{x}^i + \mathbf{r}^i$ of $(X + R)^T$, denoted by $[\mathbf{x}^i + \mathbf{r}^i]_{k-1}$.
 5. Parties compute $[x_{ji}|j]_t = [\mathbf{x}^i + \mathbf{r}^i]_{k-1} - [r_{ji}|j]_t$ locally for each $i, j \in \{1, \dots, k\}$.
-

Lemma C.9. *Protocol C.8 has a communication complexity of $\mathcal{O}(n^3)$ elements over point-to-point channels to compute a transpose of a $k \times k$ matrix with $\mathcal{O}(n^2)$ elements.*

Linear Transformation As described earlier, the linear transformation operation captures broad types of operations, such as fan-out and permutation, typically required to evaluate the circuit using the packed secret sharing. The goal of this protocol is to compute pairs of the form $(\mathbf{r}, L(\mathbf{r}))$ such that each component of both these vectors is degree- t shared among parties. We use the protocol from [20] without any major modifications. The only change is that instead of invoking their transpose protocol which uses P_{king} , we use our transpose protocol described in Protocol C.8. The protocol taken from [20] appears below.

Protocol C.10: Linear Transformation Pair Generation Protocol

- **Input:** Parties hold k linear maps $L_1, L_2, \dots, L_k$. They also hold $\{[r_{ij}|j]_t, [r_{ji}|j]_t\}_{i,j=1}^k$ computed using Protocol C.6.
 - **The protocol:**
 1. Parties compute $[L_{ji}(\mathbf{r}_j)|j]_t = L_{ji}([r_{j1}|j]_t, [r_{j2}|j]_t, \dots, [r_{jk}|j]_t)$ for all $i, j \in \{1, \dots, k\}$ locally, where L_{ji} denotes the i th component of L_j .
 2. Parties invoke the transpose protocol (Protocol C.8) on $\{[L_{ji}(\mathbf{r}_j)|j]_t\}_{i,j=1}^k$ to obtain $\{[L_{ij}(\mathbf{r}_i)|j]_t\}_{i,j=1}^k$.
 3. Parties compute their output as $\{([r_{ij}|j]_t, [L_{ij}(\mathbf{r}_i)|j]_t)\}_{i,j=1}^k$.
-

Lemma C.11. *Protocol 3.8 has a communication complexity of $\mathcal{O}(n^3)$ elements over point-to-point channels to compute random pairs corresponding to k linear transformations on k -length vectors, that is a total of $\mathcal{O}(n^2)$ elements.*

Preparing Beaver Triples As discussed, our protocol requires that parties hold degree- t sharing of random Beaver triples [7] which can be packed on demand as required during the protocol. For this, we use the existing protocol of [2] which incurs an amortized cost of $\mathcal{O}(n)$ per triple. The functionality for triple generation appears below.

Functionality C.12: $\mathcal{F}_{\text{Triples}}$ - Beaver Triples Functionality

The functionality is parameterized by the set of corrupted parties $\mathcal{C} \subseteq [n]$.

1. The functionality receives the public parameters p and ℓ .
 2. For each $k \in \{1, \dots, p\}$, the functionality does the following:
 - (a) It samples random elements $x, y \in \mathbb{F}$ and computes $z = xy$. Further, it samples random degree- t polynomials $f(x), g(x), h(x)$ such that $f(-\ell) = x, g(-\ell) = y$ and $h(-\ell) = z$.
 - (b) The functionality then sends $(f(j), g(j), h(j))$ to each party P_j .
-

Lemma C.13 ([9]). *There exists a protocol which realises $\mathcal{F}_{\text{Triples}}$ and has a communication complexity of $\mathcal{O}(np + n^5)$ elements over point-to-point channels to share p random Beaver triples via degree- t Shamir sharing.*

¹⁶ Using n instances of the private reconstruction protocol, one towards each party.

C.3 Online Phase

VSS without Privacy We now describe the VSS protocol from [12] modified for sharing non-private values.

Protocol C.14: $(k-1, k-1)$ VSS Protocol

- **Input:** The dealer holds a bivariate polynomial $S(x, y)$ of degree $k-1$ in x and y . Other parties hold no input.
 - **The protocol:**
 1. The dealer sends (shares, $D, i, (f_i(x), g_i(y))$) with $f_i(x) = S(x, i)$ and $g_i(y) = S(i, y)$ to each party P_i .
 2. Each party P_i does as follows
 - (a) Upon receiving shares from the dealer, P_i verifies that $f_i(x)$ and $g_i(y)$ are of degree at most $k-1$. If so, P_i sends (exchange, $i, j, f_i(j), g_i(j)$) to every P_j .
 - (b) Upon receiving shares (exchange, $j, i, u_{j,i}, v_{j,i}$) from P_j , check that $u_{j,i} = g_i(j)$ and $v_{j,i} = f_i(j)$. If so, broadcast the message OK(i, j).
 3. The dealer does as follows
 - (a) Initialize an undirected graph G where $V = [n]$. Upon viewing broadcasted messages OK(r, s) broadcasted by P_r and OK(s, r) broadcasted by P_s , add the edge (r, s) to G .
 - (b) Run the STAR algorithm to find sets $(C, D) \subset [n]^2$ where $|C| \geq 2t+1$, $|D| \geq 3t+1$, and for every $c \in C$ and $d \in D$, the edge (c, d) is in G .
 - (c) Let G be the set of parties that agree with at least $2t+1$ parties in C . That is, initialize $G = \emptyset$ and add i to G if $|I(i) \cap C| \geq 2t+1$.
 - (d) Let F be the set of parties that agree with at least $3t+1$ parties in G . That is, initialize $F = \emptyset$ and add i to F if $|I(i) \cap G| \geq 3t+1$.
 - (e) If $|C| \geq 2t+1$, $C \subseteq D$, and $|D|, |G|, |F| \geq 3t+1$, then broadcast (C, D, G, F) . Otherwise, continue to listen to OK messages, and repeat.
 4. **(Verifying (C, D, G, F)):** Each party P_i :
 - (a) Initialize an undirected graph G_i where $V = [n]$. Upon viewing broadcasted messages OK(r, s) from P_r and OK(s, r) from P_s , add the edge (r, s) to G_i .
 - (b) Check if (C, D, G, F) is valid for the graph G_i . If not, then continue to listen to OK message, and with each new edge, re-check validation.
 5. Once (C, D, G, F) are valid for G_i , P_i outputs its share as follows and terminates subsequently
 - **(Parties $i \in G \cup F$)** Output $(f_i(x), g_i(y))$.
 - **(Parties $i \notin G \cup F$)**
 - (a) Consider the messages $(u_{j,i}, v_{j,i})$ from Step 2b for parties $j \in F$, and using Reed Solomon decoding, try to decode the unique univariate polynomials $f_i(x), g_i(y)$ of degree- $k-1$ satisfying $f_i(j) = v_{j,i}$ and $g_i(j) = u_{j,i}$ for all but t values in F . If there is no unique decoding, wait to receive additional points $(u_{j,i}, v_{j,i})$.
 - (b) Once decoded, output $(f_i(x), g_i(y))$.
-

Lemma C.15. *Protocol C.14 when run in the batching mode has a communication complexity of $\mathcal{O}(p + n^4)$ elements over point-to-point channels to share p secrets (without privacy).*

C.4 The Complete MPC Protocol

Protocol C.16: AMPC – Π_{MPC}

(Circuit Transformation Phase)

- Parties first apply the deterministic circuit transformation algorithm from [19] to the circuit C with depth D to obtain a circuit C' which provides packed secret sharing friendliness¹⁷. The resultant circuit C' is such that $|C'| = \mathcal{O}(|C| + k \cdot (n + D))$.

¹⁷ Alternatively, the circuit could also be transformed using the algorithm from [13] leading to a circuit C' such that $|C'| = \mathcal{O}(|C| \log |C| + nD^2 \log^3 |C|)$ and $D' = \mathcal{O}(D \log^2 |C|)$

(Preprocessing Phase)

- Parties generate $\lfloor \frac{C'}{k} \rfloor$ groups of Beaver triples of the form $\{[a_i|i]_t, [b_i|i]_t, [c_i|i]_t\}_{i \in \{1, \dots, k\}}$ by invoking $\mathcal{F}_{\text{Triples}}$ (Functionality C.12).
- Parties also generate $\lfloor \frac{2|C'|}{k} \rfloor$ linear transformation pairs using Protocol C.10 required in the complete circuit for fan-out and permutation operations.

(Online Phase)

- For every group of k inputs $\mathbf{x} = (x_1, \dots, x_k)$ belonging to a party P_s , it shares its inputs using the VSS protocol from [12] to distribute $[x_i|i]_t$ for all $i \in \{1, \dots, k\}$.
 - Parties then invoke an instance of the ACS protocol (Functionality C.1) to identify a subset of (at least) $n - t$ parties who successfully completed their sharing instances. A default value is considered as input for all the parties outside this set.
 - Following this, parties evaluate the circuit layer by layer as follows.
 - For every group of k input wires $\mathbf{x}$, parties locally obtain $[x_i|p(i)]_t$ for $i \in \{1, \dots, k\}$ and some permutation p over $\{1, \dots, k\}$.
 - For every group of k input wires $\mathbf{x}$, parties further invoke Protocol 3.8 on $[x_i|p(i)]_t$ for $i \in \{1, \dots, k\}$ to obtain $[x_i|i]_t$ for all $i \in \{1, \dots, k\}$.
 - For each group of addition gates with inputs $[x_i|i]_t$ and $[y_i|i]_t$ for $i \in \{1, \dots, k\}$, parties locally compute $[z_i|i]_t = [x_i|i]_t + [y_i|i]_t$.
 - For each group of multiplication gates with inputs $[x_i|i]_t$ and $[y_i|i]_t$ for $i \in \{1, \dots, k\}$, parties invoke Protocol 3.6 to compute $[z_i|i]_t = [x_i|i]_t * [y_i|i]_t$.
 - For the group of output wires in a layer $[z_i|i]_t$ and the linear transformation L to be applied on them, parties invoke Protocol 3.8 to obtain $[L_i(\mathbf{z})|i]_t$ for all $i \in \{1, \dots, k\}$.
 - For each group of output gates $[z_i|i]_t$ belonging to some party P_s , parties locally compute $[\mathbf{z}]_d$ and send their shares to P_s . Party P_s reconstructs its output $\mathbf{z}$ using online error correction on the received shares.
-

C.5 Proof of Theorem 3.11

We now construct a simulator SIM for the complete MPC protocol. Upon activation, the simulator invokes the adversary.

Simulator for the Offline Phase

Simulating VSS. Since we use the VSS from [12], the simulator for it and its security follows immediately from their work.

Simulating random sharing.

- The simulator begins by emulating $\mathcal{F}_{\text{VSS}}$ as follows.
 - If P_s is honest, then it chooses random values r_j^s for all $j \in \{1, \dots, k\}$ as the inputs for P_s and emulates the functionality to interact with the adversary.
 - If P_s is corrupt, then the simulator interacts with the adversary and emulates $\mathcal{F}_{\text{VSS}}$ as a functionality would run it. If the functionality terminates then from the shares of the honest parties, the simulator reconstructs the inputs r_j^s for all $j \in \{1, \dots, k\}$ used by the adversary.
- Following this, the simulator emulates $\mathcal{F}_{\text{ACS}}$. For this, when the adversary delivers shares to an honest party P_h in the simulation instance of $\mathcal{F}_{\text{VSS}}$ corresponding to some party P_s that terminates, the simulator simulates P_h sending (record, h, s) . The simulator also listens to the **record** messages sent by the adversary. Receive from the adversary the message (set, S') and check the conditions for S' as the functionality would. Simulate the functionality sending the set S upon receiving a command **receiveS()** from some party.
- When the adversary delivers the set S to some honest party in the simulation instance of $\mathcal{F}_{\text{ACS}}$, the simulator performs the local computation on behalf of the honest parties to compute $[\tilde{r}_j^1|j]_t, \dots, [\tilde{r}_j^{n-2t}|j]_t$ for each $j \in \{1, \dots, k\}$.

Simulating random (i, j) sharing.

- The simulator begins by emulating $\mathcal{F}_{\text{VSS}}$ for all the instances as in the prior case to share randomly chosen values $[r_m^s|i]_t, [r_m^s|j]_t$ for all $j \in \{1, \dots, k\}$ and $m \in \{1, \dots, n - 2t\}$ on behalf of each honest party P_s . Additionally, if the VSS simulation for a corrupt party terminates, then it computes the values shared by the adversary from the shares of honest parties.
- When the adversary delivers shares to an honest party P_h in all the simulation instances of $\mathcal{F}_{\text{VSS}}$ corresponding to some party P_s , on behalf of the honest parties, the simulator applies the hyperinvertible matrix operation locally to compute $[\tilde{r}_1^s|i]_t, \dots, [\tilde{r}_n^s|i]_t$ and $[\hat{r}_1^s|j]_t, \dots, [\hat{r}_n^s|j]_t$.
- It then simulates the honest parties sending to the adversary the shares of $[\tilde{r}_\ell^s|i]_t$ and $[\hat{r}_\ell^s|j]_t$ for every $\ell \in \mathcal{C}$. It also receives the shares corresponding to the honest parties from the adversary.
- The simulator checks if $\tilde{r}_h^s = \hat{r}_h^s$ and simulates broadcasting $\text{OK}(s, h)$ if it holds. It also listens to the broadcasts from the adversary.
- Following this, the simulator emulates $\mathcal{F}_{\text{ACS}}$ by sending (record, h, s) for a party P_s on behalf of an honest party P_h if P_h received its shares in P_s 's instances of VSS and received $\text{OK}(s, \ell)$ from at least $n - t$ parties. The simulator also listens to the **record** messages sent by the adversary. It receives from the adversary the message (set, S') and checks the conditions for S' as the functionality would.
- It simulates the functionality sending the set S upon receiving a command $\text{receiveS}()$ from some party. The simulator simulates sending the set S from $\mathcal{F}_{\text{ACS}}$ to the adversary when the adversary requests it.
- When the adversary delivers S in the simulation of $\mathcal{F}_{\text{ACS}}$ to honest parties, the simulator performs local computation on behalf of honest parties to obtain $[\tilde{r}_u^1|i]_t, \dots, [\tilde{r}_u^{n-2t}|i]_t$ and $[\hat{r}_u^1|j]_t, \dots, [\hat{r}_u^{n-2t}|j]_t$.

Simulating transpose operation.

- The simulator chooses vectors of random elements $\mathbf{x}_i, \mathbf{r}_i$ for each $i \in \{1, \dots, k\}$ and corresponding to each honest party it computes the shares of $[\mathbf{x}_i + \mathbf{r}_i]_d$.
- The simulator simulates sending the shares of honest parties to the corrupt parties by sending them to the adversary. It also receives the shares from the adversary on behalf of the honest parties and performs the local operation as per the protocol.

Simulate the linear transformation pair generation. For a batch of k linear maps, the simulator simulates the transpose protocol as above and performs the local operations as per the protocol steps.

Simulate the triple sharing. We use the triple sharing protocol from [2] and the existence of a simulator and its security follows immediately. Hence, the simulator emulates $\mathcal{F}_{\text{Triples}}$ as a functionality would run it.

Simulator for the Online Phase

Simulating the input phase. For each group of k input gates belonging to a party P_s , the simulator emulates the functionality $\mathcal{F}_{\text{VSS}}$ and does the following:

- If P_s is honest, then it chooses random values as the inputs for P_s and emulates the functionality to interact with the adversary.
- If P_s is corrupt, then the simulator interacts with the adversary and emulates $\mathcal{F}_{\text{VSS}}$ as a functionality would run it. If the functionality terminates then from the shares of the honest parties, the simulator reconstructs the inputs used by the adversary.

Simulating ACS.

- The simulator emulates the functionality $\mathcal{F}_{\text{ACS}}$ as follows. When the adversary delivers shares to an honest party P_h in the simulation instance of $\mathcal{F}_{\text{VSS}}$ corresponding to some party P_s that terminates, the simulator simulates P_h sending (record, h, s) .
- The simulator also listens to the **record** messages sent by the adversary.
- Simulate the functionality to send the set S upon receiving a command $\text{receiveS}()$ from some party.

Simulating degree reduction.

- The simulator chooses k vectors of k random elements each $\mathbf{u}^1, \dots, \mathbf{u}^k$ and computes the shares of honest parties corresponding to $[\mathbf{u}^1]_{d+k-1}, \dots, [\mathbf{u}^k]_{d+k-1}$.
- It defines the vector of polynomials $\mathbf{P}(x)$ and computes the shares of honest parties on this polynomial locally.
- The simulator sends share of $\mathbf{P}(i)$ corresponding to each $i \in \mathcal{C}$ to the adversary and receives from the adversary the shares $\mathbf{P}(h)$ corresponding to each honest party P_h .
- Following this, it simulates the steps of Protocol C.14 by acting as the dealer on behalf of each honest party to share $\mathbf{P}(h)$ and by emulating the role on honest parties in every instance in which a corrupt party is the dealer.
- The simulator further simulates $\mathcal{F}_{\text{ACS}}$ by sending (record, s, h) on behalf of every honest party P_h to which the adversary delivers a share in the VSS where P_s acts as the dealer. The simulator also listens to the **record** messages sent by the adversary. Receive from the adversary the message (set, S') and check the conditions for S' as the functionality would. Simulate the functionality sending the set S upon receiving a command **receiveS**() from some party.
- The simulator records the shares of honest parties corresponding to $[\mathbf{u}^1]_{k-1}, \dots, [\mathbf{u}^k]_{k-1}$.

Simulating Multiplication.

- For the group of k multiplication gates, the simulator simulates degree reduction as described above to simulate the steps for reducing $[\mathbf{x}^i + \mathbf{a}^i]_d$ and $[\mathbf{y}^i + \mathbf{b}^i]_d$ to $[\mathbf{x}^i + \mathbf{a}^i]_{k-1}$ and $[\mathbf{y}^i + \mathbf{b}^i]_{k-1}$ for all $i \in \{1, \dots, k\}$.
- It simulates degree reduction again to simulate reducing $[\mathbf{z}^i + \mathbf{r}^i]_{d+k-1}$ to $[\mathbf{z}^i + \mathbf{r}^i]_{k-1}$ for all $i \in \{1, \dots, k\}$.

Simulating Linear Transformation. For a group of k packed gates requiring the same linear transformation L to be applied, the simulator simulates the degree reduction as described above in a batched manner.

Simulating the output phase.

- The simulator invokes the functionality $\mathcal{F}_{\text{MPC}}$ with the corrupted parties' inputs computed during the input phase.
- For every group of k output gates $\mathbf{z} = (z_1, \dots, z_k)$ belonging to each corrupt party $i \in \mathcal{C}$, the simulator receives the output $\mathbf{z}$ from $\mathcal{F}_{\text{MPC}}$.
- Based on the shares of $[z_j|j]_t$ corresponding to the corrupt parties, the simulator computes the sharing $[z_j|j]_t$ and $[\mathbf{z}]_d$. On behalf of the honest parties, it sends to the adversary the shares of $[z_j|j]_t$ held by the honest parties. It also receives from the adversary the shares corresponding to the output gates belonging to honest parties.
- Finally, the simulator outputs what the adversary outputs, and the honest parties output the values received from the functionality.

We now show that the **SIM** perfectly simulates the honest parties' behaviour, as well as the functionalities $\mathcal{F}_{\text{VSS}}$, $\mathcal{F}_{\text{Triples}}$ and $\mathcal{F}_{\text{ACS}}$. It suffices to discuss this for parts of the protocol which involve communication with the corrupt parties. Moreover, we show that the output of honest parties is the same in both the executions, and in particular, each party receives its output at the same "time" in both executions. We note that the point at which each party receives its output is determined by the adversary in the real world, or the simulator (instructed by the adversary) in the ideal world (which in turn the simulator forwards to the functionality in the ideal world).

1. Offline Phase

- For simulating the VSS when the dealer is honest, **SIM** chooses random degree- t polynomials on behalf of honest parties and sends to the adversary shares on these polynomials. Since the adversary has at most t shares, the secret is uniformly random from the adversary's view in both the real and the ideal world. When the dealer is corrupt, **SIM** receives the polynomial from the adversary, and if it satisfies the conditions, **SIM** records the input provided by the adversary to the VSS. It sends to the adversary points on the same polynomial, ensuring that the distribution of shares held by the corrupt parties is identical in both the worlds.

- For simulating random sharing, upon simulating $\mathcal{F}_{\text{VSS}}$ perfectly, **SIM** sends to the adversary the **record** messages identically to the real world. Moreover, it receives from the adversary the set S' and verifies its conditions before sending it to all the parties as in the real world. This ensures that the adversary's view is identically distributed in both the real and ideal world.
- To simulate random (i, j) sharing, **SIM** begins by simulating $\mathcal{F}_{\text{VSS}}$ by choosing random values on behalf of the honest parties, ensuring the adversary's view is distributed exactly as in the real world. Moreover, it learns the honest parties' shares on the polynomials distributed by the corrupt parties. **SIM** sends the honest parties' shares corresponding to $[\hat{r}_\ell^s[j]]_t$ and $[\hat{r}_\ell^s[j]]_t$ for all corrupt parties P_ℓ to the adversary. Since the values getting reconstructed to the adversary and uniformly random, the distribution of adversary's view is identical in both the worlds. It also receives from the adversary the shares on behalf of honest parties and broadcasts the **record** messages identically to the broadcasts of honest parties in the real world. The simulation of $\mathcal{F}_{\text{ACS}}$ is again as before.
- For simulating the transpose operation, the values $\mathbf{x}_i + \mathbf{r}_i$ reconstructed by **SIM** to the adversary are uniformly random since $\mathbf{r} = (r_1, \dots, r_k)$ is uniformly random given the t shares each of $r_1, \dots, r_k$ that the adversary holds. Hence this does not influence the adversary's view.
- $\mathcal{F}_{\text{Triples}}$ sends to the adversary at most t shares corresponding to random triples that it chooses on behalf of the honest parties. Given the properties of degree- t Shamir sharing, it is ensured that each component of the triple is uniformly random from the view of the adversary.

2. Online Phase

- During the input stage, **SIM** chooses random inputs on behalf of honest parties and sends to the adversary (at most) t shares corresponding to the corrupt parties. Again, given the properties of Shamir sharing using a degree- t polynomial, the adversary's view is independent of the secrets chosen by the simulator. The simulator also learns the inputs used by the corrupt parties. These inputs are eventually used to invoke $\mathcal{F}_{\text{MPC}}$.
- **SIM** simulates $\mathcal{F}_{\text{ACS}}$ by sending **record** messages based on the simulation of $\mathcal{F}_{\text{VSS}}$ as described earlier. This does not influence the distribution of the adversary's view. The set S obtained from the adversary in $\mathcal{F}_{\text{ACS}}$ retained for invoking $\mathcal{F}_{\text{MPC}}$.
- Degree reduction is invoked from the perspective of multiplication for reducing $[\mathbf{x}^i + \mathbf{a}^i]_d$ and $[\mathbf{y}^i + \mathbf{b}^i]_d$ to $[\mathbf{x}^i + \mathbf{a}^i]_{k-1}$ and $[\mathbf{y}^i + \mathbf{b}^i]_{k-1}$ respectively. It is also used to reduce $[\mathbf{z}^i + \mathbf{r}^i]_{d+k-1}$ to $[\mathbf{z}^i + \mathbf{r}^i]_{k-1}$ for all $i \in \{1, \dots, k\}$. The vectors $\mathbf{a}^i = (a_1^i, \dots, a_k^i)$, $\mathbf{b}^i = (b_1^i, \dots, b_k^i)$, $\mathbf{r}^i = (r_1^i, \dots, r_k^i)$ are such that each component is degree- t shared and the adversary knows at most t shares. Thus, these vectors of values are uniformly random from the adversary's view. Hence, during degree reduction, the t linear combinations of $(\mathbf{u}^1, \dots, \mathbf{u}^k) = (\mathbf{a}^1, \dots, \mathbf{a}^k)$ reconstructed towards the adversary are also uniformly random.
- Similarly, degree reduction is used to reduce $[\mathbf{x} + \mathbf{r}]_d$ to $[L(\mathbf{x} + \mathbf{r})]_{k-1}$. Again, since $\mathbf{r}$ is uniformly random from the view of the adversary, $\mathbf{x} + \mathbf{r}$ is random and so is a linear combination of many such vectors.
- Finally, in the output phase, **SIM** invokes $\mathcal{F}_{\text{MPC}}$ with the inputs extracted from the adversary during the input phase and a set of (at least) $n - t$ parties received from the adversary during the simulation of $\mathcal{F}_{\text{ACS}}$. It receives the outputs corresponding to the corrupt parties from the functionality. Finally, for each output $\mathbf{x}$ corresponding to a corrupt party, **SIM** uses the shares of $[\mathbf{x}]_d$ corresponding to the corrupt parties that it recorded from the prior stages and $\mathbf{x}$ to compute the honest parties' shares of $[\mathbf{x}]_d$. It then sends the shares to the adversary on behalf of the honest parties. This ensures that the distribution of the adversary's view remains the same as that in the real world. Moreover, honest parties receive their outputs from $\mathcal{F}_{\text{MPC}}$ which is invoked on the inputs of corrupt parties extracted by **SIM** and the set of (at least) $n - t$ parties recorded during the $\mathcal{F}_{\text{ACS}}$ invocation, ensuring the correctness of their output. Moreover, since the adversary decides the scheduling of when the honest parties receive their output (via the simulator), and the view of the adversary is exactly the same in the real and ideal world, the scheduling is exactly the same. Hence, honest parties receive their output at exactly the same time in both, the real and the ideal world.

Analysis of the communication complexity In the offline phase, parties invoke the triple generation protocol from [9] which incurs a cost of $\mathcal{O}(C'n + n^5)$ for generating shares corresponding to C' number

of triples. Additionally, parties also compute a linear transformation pair corresponding to each packed gate in the circuit. Our protocol for linear transformation pair generation incurs a cost of $\mathcal{O}(n^3)$ for generating the pairs corresponding to $\mathcal{O}(n)$ distinct linear transformations corresponding to $\mathcal{O}(n)$ packed gates. Since each packed gate evaluates $\mathcal{O}(n)$ gates simultaneously, this amounts to a cost of $\mathcal{O}(n)$ per gate in the circuit. Thus, linear transformation pair generation incurs a cost of $\mathcal{O}(C'n + n^3)$. In total, the preprocessing phase has a cost of $\mathcal{O}(C'n + n^5)$.

During the online phase, parties begin by sharing their inputs to the circuit. Given that the VSS from [12] requires $\mathcal{O}(np + n^4)$ to share p secrets, and all the n parties may require to share their inputs, this amounts to a cost of $\mathcal{O}(nI + n^5)$, where I is the number of inputs to the circuit. Further, the ACS invocation taken from the work of [3] incurs a one time cost of $\mathcal{O}(n^5)$. Further, to prepare the inputs to each layer of the circuit, parties apply the linear transformation operation a constant number of times per packed gate. The first time is for creating the required fan-out for each value. It is used again to ensure that the secrets required as input to a gate lie at different positions in their respective sharings. And finally a permutation is used to order the secrets within a permutation in the required order. Given that the linear transformation operation incurs a cost of $\mathcal{O}(p + n^5)$ for p gates (or equivalently p/k packed gates), and that this operation is performed at each layer of the circuit, this operation incurs a cost of $\mathcal{O}(|C'| + Dn^5)$ for the complete circuit. Next, the multiplication gates incur a cost of $\mathcal{O}(p + n^5)$ for evaluating p gates, thus requiring a total of $\mathcal{O}(|C'|n + Dn^5)$ to evaluate the circuit completely. Finally, reconstruction of an output towards a single party incurs a cost of $\mathcal{O}(n)$, requiring a cost of $\mathcal{O}(n^2O)$ to reconstruct all the outputs towards all parties. Here, O denotes the number of output gates. In conclusion, the protocols incurs a cost of $\mathcal{O}(|C'|n + Dn^5 + n^5)$ in the online phase and $\mathcal{O}(n^2O)$ in the output phase.

D 2-Round Perfect MPC with Abort Security

In this section, we prove the sufficiency of 2 rounds for achieving perfectly secure computation with abort security. While the impossibility of 1-round protocols for abort security was already known from existing results, to the best of our knowledge, the lower bound results have only considered GOD to establish a bound of 4 rounds for generic secure computation [6]. Consequently, the question of whether the 4-round lower bound also applies for abort security, or weakening the security from GOD to abort benefits the optimal round complexity, has been open. Our work shows that indeed we can do better for abort security by proving that 2 rounds are sufficient to achieve perfect security with abort.

D.1 2-Round Verifiable Secret Sharing

Our first primitive is a 2-round VSS protocol which offers abort security. The protocol itself is very simple with the first round comprising of the dealer sending shares of its chosen degree- (t, t) symmetric bivariate polynomial $S(x, y)$ to each party. Consequently, the share of each party P_i is a univariate polynomial $S(x, i)$. Additionally, in the same round, every party also chooses a degree- t random polynomial $r_i(x)$ and sends one point on the polynomial to each party. Looking ahead, this polynomial will be used for masking P_i 's share of $S(x, y)$ in order enable performing the pairwise consistency check publicly in the next round. The second round consists of every party broadcasting its entire masked share polynomial together with the masked shares of the univariate share polynomial of every other party. Following this, parties locally check the pairwise consistency of all broadcasted values and abort in case of an inconsistency. The functionality and the corresponding protocol appear below. The proof is deferred to Section E.

Functionality D.1: VSS Protocol with Abort Security

The functionality is parameterized by a set of corrupted parties $\mathcal{C} \subseteq [n]$. **Input:** The dealer holds a secret s and a (t, t) -degree symmetric bivariate polynomial $S(x, y)$ such that $S(0, 0) = s$. Other parties do not have any input.

1. The functionality receives a bivariate polynomial $S(x, y)$ from the dealer. If the polynomial has degree higher than t in either x or y then it ignores the higher order terms to ensure a polynomial of degree less than or equal to t in both variables.

2. If the dealer is honest, the functionality sends to the adversary $S(x, i)$ for each $i \in \mathcal{C}$.
 3. It receives from the adversary either **abort** or **continue**.
 4. If **continue** is received, and the bivariate polynomial is a symmetric then it sends $S(x, i)$ to all parties. Otherwise, it sends $\perp$ to all.
-

Protocol D.2: 2-Round VSS Protocol

- **Input:** A party P_s holding a secret s to be shared via degree- t Shamir-sharing.
 - **The protocol:**
 1. **(Round 1)**
 - (a) P_s chooses a (t, t) -degree symmetric bivariate polynomial $S(x, y)$ such that $S(0, 0) = s$.
 - (b) P_s sends to each P_i the polynomial $f_i(x) = S(x, i)$. Each P_i sets its share as $s_i = f_i(0)$.
 - (c) Each P_i chooses a random polynomial $r_i(x)$ of degree at most t and sends $r_{ij} = r_i(j)$ to every P_j .
 2. **(Round 2)**
 - (a) For each ordered pair (i, j) , parties P_i and P_j broadcast $m_i(x) = f_i(x) + r_i(x)$ and $m_{ij} = f_j(i) + r_{ij}$ respectively.
 3. **(Local Computation)** If some $m_i(x)$ has degree more than t or there exists some (i, j) such that $m_i(j) \neq m_{ij}$, then parties abort and output $\perp$.
-

Theorem D.3. *Let $n = 3t + 1$. Protocol D.2 computes Functionality D.1, $\mathcal{F}_{\text{VSS-abort}}$, in the presence of a malicious adversary in a synchronous network.*

D.2 2-Round Multiplication

The completeness of quadratic functions, that is degree-2 functions, has already been settled in a series of work [4,5,6]. Moreover, it was shown in [6, Proposition 4.5] that for perfect security, a protocol for a specific category of degree-2 functions implies a protocol for all degree-2 functions. In particular, they showed that a protocol for the following function suffices to prove the existence of a protocol for all degree-2 functions: $f : x^\alpha x^\beta + \sum_j r^j$ where x^α, x^β correspond to inputs of some parties P_α, P_β respectively and r^j corresponds to the input of P_j . Below, we provide the functionality and a 2 round protocol for computing it, which provides abort guarantees. The proof appears in Section E.

Functionality D.4: Degree-2 Computation – $\mathcal{F}_{\text{D2C}}$

Input: P_α, P_β hold $x^\alpha, x^\beta \in \mathbb{F}$ respectively. Each P_j holds input $r^j \in \mathbb{F}$.

1. The functionality receives x^α, x^β from P_α, P_β respectively. It also receives r^j from each P_j . It sends $v = x^\alpha x^\beta + \sum_j r^j$ to the adversary.
 2. The functionality receives from the adversary either **abort** or **continue**.
 3. If **continue** is received, it computes $v = x^\alpha x^\beta + \sum_j r^j$ sends it to all the parties. Otherwise, it sends $\perp$ to all.
-

Protocol D.5: 2-Round Degree-2 Computation Protocol

- **Input:** Parties P_α, P_β hold x^α, x^β respectively. Every party P_j holds r^j .
- **Goal:** The goal is to compute $v = x^\alpha x^\beta + \sum_j r^j$.
- **The protocol:**
 1. **(Round 1)**
 - (a) P_α, P_β each initiates an instance of the VSS protocol, and all the parties perform **Round 1** computation of these instances. Let $[x^\alpha]_t^i, [x^\beta]_t^i$ denote the share received by P_i .

- (b) Simultaneously, each P_j initiates an instance of VSS to share r^j and parties also execute the **Round 1** computation of these instances. Let $[r^j]_t^i$ denote the share received by P_i .
2. (**Round 2**)
- (a) Parties execute the **Round 2** computation of all the VSS instances. If any instance aborts, then parties abort and output $\perp$.
- (b) Each P_i computes $[v]_{2t}^i = [x^\alpha]_t^i [x^\beta]_t^i + \sum_j [r^j]_t^i$ and broadcasts it.
3. (**Local Computation**) If $\{[v]_{2t}^i\}_{i \in [n]}$ are such that they all lie on a unique degree- $2t$ polynomial say $f_v(x)$, then parties compute $f_v(0)$ as the output. Otherwise, parties abort and output $\perp$.
-

Theorem D.6. *Let $n = 3t + 1$. Protocol D.5 computes Functionality D.4, $\mathcal{F}_{\text{D2C}}$, in the presence of a malicious adversary in a synchronous network¹⁸.*

E Proofs for 2-Round Perfectly-Secure Protocol with Abort Security

E.1 Proof of Theorem D.3

Proof. There are two cases to consider.

1. When the dealer is honest:

- The simulator receives from the functionality the polynomials $S(x, i)$ for each corrupt party $i \in \mathcal{C}$ and simulates the dealer sending these polynomials to each corrupt P_i . It also chooses a random (t, t) -degree symmetric bivariate polynomial $S'(x, y)$ such that $S'(x, i) = S(x, i)$ received from the functionality.
- It also chooses random degree- t polynomials $r_j(x)$ corresponding to each honest P_j and simulates P_j sending r_{ji} to each corrupt P_i . It receives from the adversary the points r_{ij} on behalf of every honest party P_j .
- The simulator broadcasts $m_j(x) = S'(x, j) + r_j(x)$ and $m_{ji} = m_j(i)$ on behalf of all honest parties P_j . It also listens to adversary's broadcasts.
- If there exists some $m_i(x)$ has degree more than t or some i, j such that $m_i(j) \neq m_j(i)$ then, on behalf of the adversary, the simulator sends **abort** to the functionality and receives $\perp$ from it. Otherwise, it sends **continue**, receives the shares $S(x, i)$ for each corrupt party $i \in \mathcal{C}$ and sends them to the adversary.
- The simulator outputs whatever the adversary outputs and terminates.

2. When the dealer is corrupt:

- The simulator receives from the adversary the shares $S(x, j)$ on behalf of each honest party P_j . It interpolates the polynomial $S(x, y)$ from the shares of arbitrary $t + 1$ honest parties and sends it to the functionality.
- It chooses random degree- t polynomials $r_j(x)$ corresponding to each honest P_j and simulates P_j sending r_{ji} to each corrupt P_i . It receives from the adversary the points r_{ij} on behalf of every honest party P_j .
- The simulator broadcasts $m_j(x) = S(x, j) + r_j(x)$ and $m_{ji} = m_j(i)$ on behalf of all honest parties P_j . It also listens to adversary's broadcasts.
- If there exists some $m_i(x)$ has degree more than t or some i, j such that $m_i(j) \neq m_j(i)$ then, on behalf of the adversary, the simulator sends **abort** to the functionality. It then receives $\perp$ from the functionality and sends it to the adversary. Otherwise, it sends **continue**, receives the shares $S(x, i)$ for each corrupt party $i \in \mathcal{C}$ and sends them to the adversary.
- The simulator outputs whatever the adversary outputs and terminates.

We now show that the simulator perfectly simulates the honest parties' behaviour. We discuss this for the parts of the protocol which involve communication with the corrupt parties.

1. Round 1

¹⁸ The same holds true for $t < n/3$ by modifying the protocol to allow only the first $3t + 1$ parties to broadcast their $[v]_{2t}$ shares in Round 2 of the protocol.

- For simulating the round 1 messages when the dealer is honest, the simulator receives the univariate polynomial shares from the functionality which correspond to the dealer's correct polynomial, and hence the adversary's view in the real and ideal world is identical. Moreover, the adversary receives at most t shares. Given the properties of Shamir sharing that the secret is uniformly random given the corrupt parties' share, this does not affect the distribution of honest parties' output and corrupt parties' shares.
- Further, the adversary receives at most t shares on the degree- t random polynomials $r_j(x)$ chosen by each honest party P_j in the real world. In the ideal world, the simulator chooses the $r_j(x)$ polynomials on behalf of the honest parties from the same distribution as the honest parties would. By the same argument as above, this does not affect the distribution of honest parties' output and the corrupt parties' shares.

2. Round 2

- If the dealer is honest, then by the properties of Shamir sharing, it is guaranteed that the dealer's secret is uniformly random from the view of the adversary even after it receives t shares $S(x, i)$ corresponding to each $i \in \mathcal{C}$. Hence, the polynomial $S'(x, y)$ chosen by the simulator, and consequently the polynomials $m_j(x) = S'(x, j) + r_j(x)$ are identically distributed from the adversary's view in both the worlds.
- If the dealer is corrupt, then the simulator computes $m_j(x)$ and m_{ji} s for each honest P_j using precisely the $S(x, j)$ shares received from the adversary and the randomly chosen polynomial $r_j(x)$ which is distributed identically from the adversary's view.

3. **Local Computation** If none of the honest parties abort in the simulated execution, it means that all the honest parties are pairwise consistent in the check performed in round 2. Hence the shares $S(x, j)$ corresponding to any $t + 1$ honest parties define the unique bivariate polynomial used by a corrupt dealer. Simulator invokes the functionality with precisely this polynomial extracted from the shares of the honest parties, and further sends **continue** thus ensuring that the view of honest parties is the same in both the worlds. Otherwise, the simulator sends the command **abort** to the functionality, again ensuring that the honest parties receive $\perp$ in the ideal world as well.

□

E.2 Proof of Theorem D.6

Proof. We now construct a simulator for the degree two computation protocol.

- The simulator chooses a random r^j on behalf of each honest party P_j and emulates round 1 of the VSS protocol as described in the simulation for VSS for the case of an honest dealer. Similarly, if P_α (resp P_β) is honest, then the simulator chooses x^α (resp. x^β) and emulates round 1 of VSS for an honest dealer case.
- It also receives from the adversary the round 1 shares corresponding to r^i for each $i \in \mathcal{C}$, and of x^α (resp. x^β) if P_α (resp. P_β) is corrupt. Consequently, from the shares of the honest parties, the simulator learns the bivariate polynomial corresponding to each VSS instance and thus extracts r^i for all $i \in \mathcal{C}$ and x^α (resp. x^β) if P_α (resp. P_β) is corrupt.
- Let the sharing generated be represented by $[r^j]$ and $[x^\alpha], [x^\beta]$.
- The simulator sends to the functionality r^i for each $i \in \mathcal{C}$. It additionally sends x^α (resp. x^β) if P_α (resp. P_β) is corrupt. It receives $v = x^\alpha x^\beta + \sum_j r^j$ from the functionality.
- It emulates round 2 of VSS as described in its simulation, and decides on **abort** or **continue** for each instance of the VSS. Additionally, it also broadcasts $[v]^j = [x^\alpha]^j [x^\beta]^j + \sum_k [r^k]^j$ on behalf of every honest P_j . It also listens to the adversary's broadcasts.
- If any VSS simulation results in **abort**, or the broadcasted shares do not lie on a unique degree- $2t$ polynomial, then it sends **abort** to the functionality. Otherwise, it sends **continue** and receives v from it.
- The simulator sends v to the adversary and outputs whatever it outputs.

We now show that the simulator perfectly simulates the honest parties' behaviour. We discuss this for the parts of the protocol which involve communication with the corrupt parties.

1. Round 1

- For simulating the round 1 messages for every honest party P_j acting as the dealer, the simulator chooses a random element r^j and a corresponding bivariate polynomial to initiate VSS as the honest party would. The adversary receives t univariate polynomial shares on these bivariate polynomials. Given the properties of Shamir sharing that the secrets are uniformly random given the corrupt parties' share, this does not affect the distribution of honest parties' output and corrupt parties' shares. The same argument works when P_α (resp. P_β) are honest parties.

2. Round 2

- If the dealer is honest, then by the properties of Shamir sharing, it is guaranteed that the dealer's secret is uniformly random from the view of the adversary even after it receives t shares corresponding to each $i \in \mathcal{C}$. Hence, the bivariate polynomial chosen by the simulator, and consequently the polynomials broadcasted by it in round 2 simulation of the VSS instances are identically distributed from the adversary's view in both the worlds.
- If the dealer is corrupt, then the simulator computes the values to be broadcasted on behalf of each honest P_j in round 2 of VSS using precisely the shares received from the adversary and the randomly chosen polynomial which is distributed identically in both the worlds from the adversary's view.
- Further, the $[v]^j$ broadcasted by the simulator on behalf of each honest P_j is computed as a linear combination of shares received from the adversary and the shares assumed on behalf of other honest parties corresponding to randomly chosen secrets. By the linearity property of Shamir sharing and the property discussed above, the view of the adversary remains identically distributed in the real and ideal world.

3. **Local Computation** If none of the honest parties abort in the simulated execution, it means that all the honest parties are pairwise consistent in the check performed in round 2 of the VSS. Hence the shares corresponding to any $t + 1$ honest parties define the unique bivariate polynomials used by the corrupt dealers. Simulator invokes the functionality with precisely these polynomials extracted from the shares of honest parties. Further, no party aborting also implies that the shares of $[v]$ broadcasted by all the parties indeed lie on a degree- $2t$ polynomial and thus v is output by all in the simulation. The simulator sends `continue` when no party aborts, thus ensuring that honest parties receive v in the ideal world. Consequently, their view is the same in both the worlds. Otherwise, the simulator sends the command `abort` to the functionality, again ensuring that the honest parties receive $\perp$ in the ideal world as well.

□